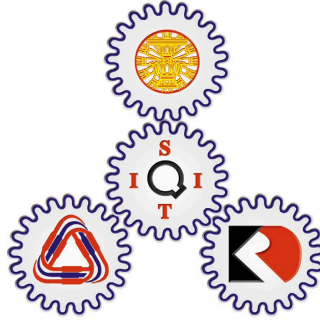




Database System Design

Kittin Traisiwakul	6622780409
Thinnaphop Direkpok	6622781969
Jetnipat Wanichsakul	6622780318
Kanade Areepoonsiri	6622770442
Patipat Permsub	6622780151
Pisit Pongpan	6622781852

CSS325: Database System

Asst. Prof. Dr. Preecha Tangworakitthaworn

SIRINDHORN INTERNATIONAL INSTITUTE OF TECHNOLOGY

THAMMASAT UNIVERSITY

November, 2025

Table of Contents

Reason why we have 4 entities	3
Revised ERD	4
Mini ERD and Revised Mini ERD	5
Relational Diagram - 8 Steps Transformation	6
Final relational diagram	10
Pre-Processing	11
Data Dictionary	21
Create a database in MAMP server (DDL)	43
Example of Data Manipulation Language (DML)	51
Random data record using Python	58
SQL queries	84
References	111

Reason why we have 4 entities and 1 relationship (M:N) from Kaggle

First submission: 30 Aug - selecting data from kaggle

First, we selected this Kaggle file because it consists of 7 (.csv) files (aisle, departments, order_products_prior, order_products_train, orders, products, and sample_submission). The topic itself is interesting, and we thought that it qualified the criteria of having 3 master files and 2 transaction files.

After the first submission, we got feedback from the professor that we chose a “Good dataset”, and by that time, we still lacked the knowledge to be able to point out the flaws in the Kaggle dataset. As per that time, we still have not learned about how to draw relationship-schema yet, which includes Primary Key and Foreign Key. Furthermore, the erd/erd we practiced at that time is still a basic concept of knowing the relationship between entities, but not yet to be implemented on a big scale.

Second submission: 20 Sep - drawing erd/erd

By the time we do the second submission, we start writing a blueprint for our business project by drawing erd/erd diagram. The erd/erd diagram is being designed by ourselves according to the business rule we determined.

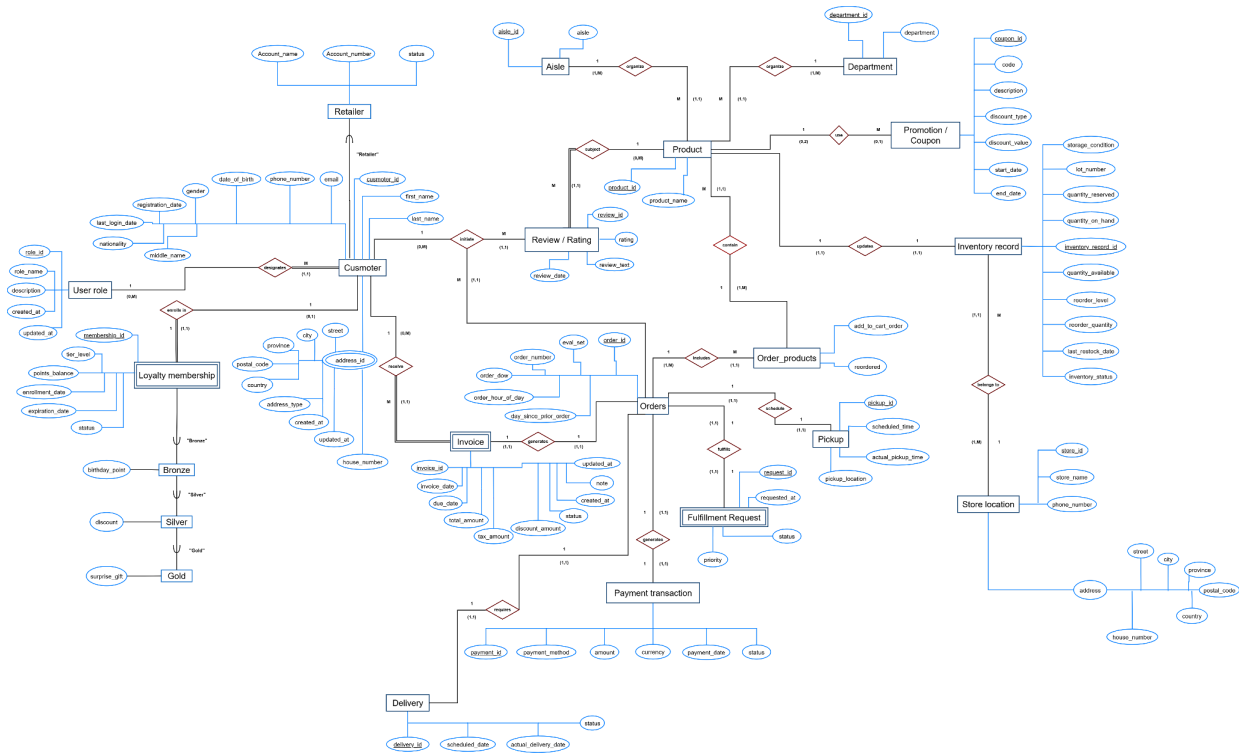
According to the “Good dataset” feedback, we all thought that our data is good to go and they are all entities. Hence, we designed the erd/erd diagram while considering the 5 files from Kaggle as entities, so as the relationship between each of them.

Third submission: 18 October - drawn schema, didn't check the kaggle's data

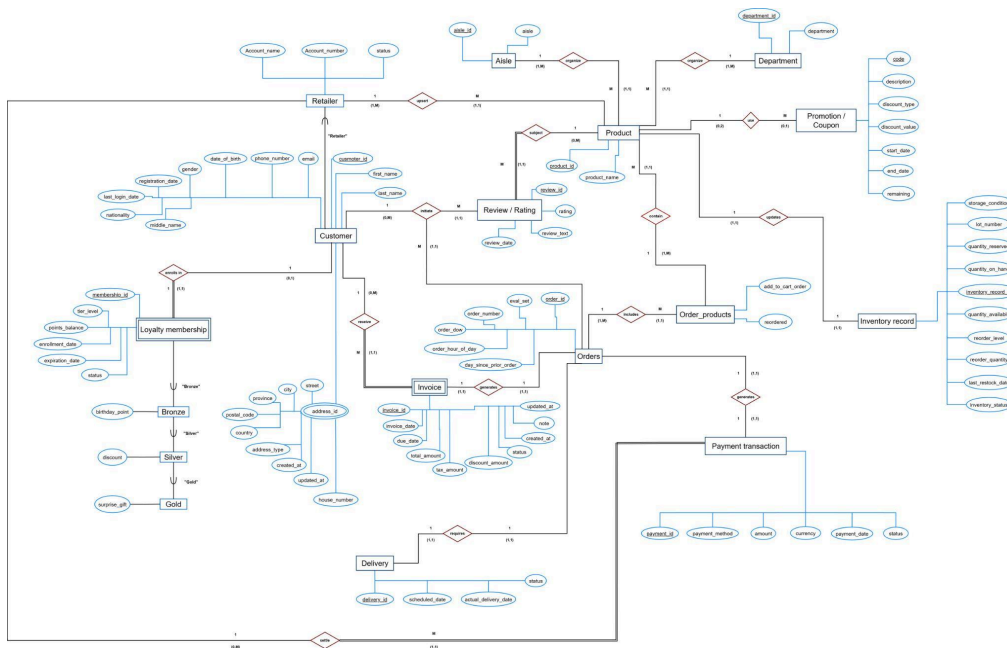
In our third submission, we are now implementing the 8 steps relational-schema to our project. However, as we have the idea that the 5 Kaggle files we chose from phase 1 are all entities. We do the relational-schema according to the mini erd diagram we designed, which considered that the 5 Kaggle files are all entities.

This results in us not having to go back and check the original Kaggle file until we start doing the *fourth submission: 1 November*, which we need to develop the data dictionary based on our relational database schema. In doing this fourth submission, we have to load and insert data from the Kaggle dataset that is selected from phase 1. For this reason, we went back and checked the 5 Kaggle files thoroughly and found out that one of the 5 Kaggle files we chose is an associatived entity, which resulted from an many-to-many relationship between 2 Kaggle entities (files).

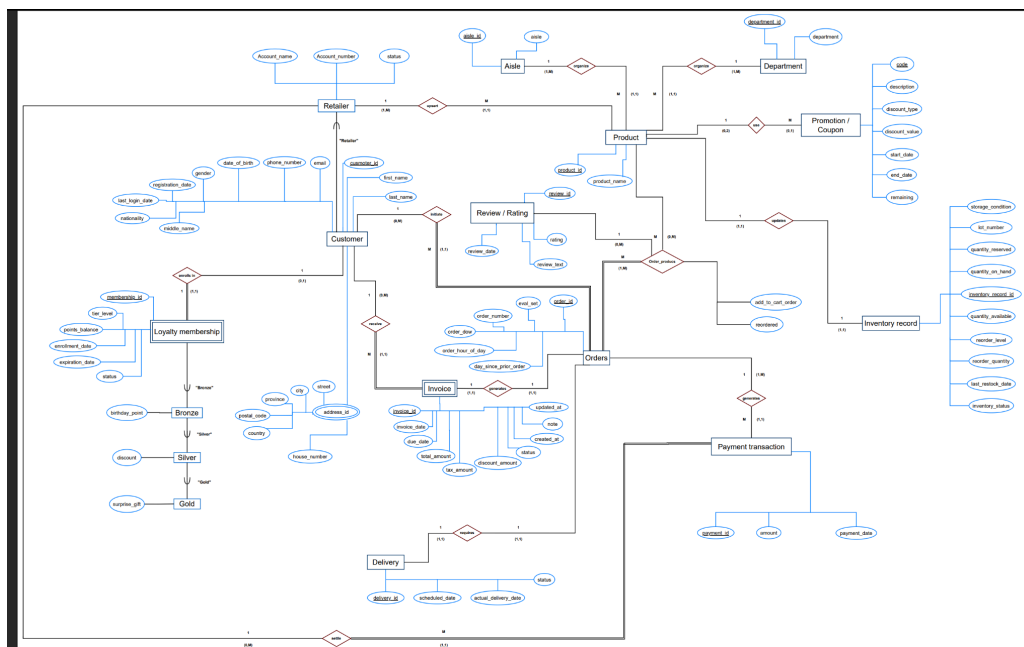
Revised ERD/EERD



Mini ERD/EERD Diagram



Revised Mini ERD and Revised Relation Schema



Relational Diagram - 8 Steps Transformation

Step 1: Regular Entity

Step 1 : Strong

Aisle

<u>aisle_id</u>	aisle
-----------------	-------

Department

<u>department_id</u>	department
----------------------	------------

Order_products

<u>add_to_cart_order</u>	recorded
--------------------------	----------

Orders

<u>order_id</u>	eval_set	order_number	order_dow	order_hour_of_day	day_since_prior_order
-----------------	----------	--------------	-----------	-------------------	-----------------------

Product

<u>product_id</u>	product_name
-------------------	--------------

Promotion

<u>code</u>	description	discount_type	discount_value	start_date	end_date	remaining
-------------	-------------	---------------	----------------	------------	----------	-----------

Payment Transaction

<u>payment_id</u>	payment_method	amount	currency	payment_date	status
-------------------	----------------	--------	----------	--------------	--------

Customer

<u>customer_id</u>	first_name	last_name	email	phone_number	date_of_birth	gender	registration_date	last_login_date	nationality	middle_name
--------------------	------------	-----------	-------	--------------	---------------	--------	-------------------	-----------------	-------------	-------------

Inventory Record

<u>inventory_record_id</u>	quantity_on_hand	quantity_reserved	quantity_available	reorder_level	reorder_quantity	last_restock_date	inventory_status	lot_number	storage_condition
----------------------------	------------------	-------------------	--------------------	---------------	------------------	-------------------	------------------	------------	-------------------

Delivery

<u>delivery_id</u>	schedule_date	actual_delivery_date	status
--------------------	---------------	----------------------	--------

Review/Rating

<u>review_id</u>	rating	review_text	review_date
------------------	--------	-------------	-------------

Step 2: Weak Entity

Step 2 : Weak

Customer

<u>customer_id</u>	first_name	last_name	email	phone_number	date_of_birth	gender	registration_date	last_login_date	nationality	middle_name
--------------------	------------	-----------	-------	--------------	---------------	--------	-------------------	-----------------	-------------	-------------

Orders

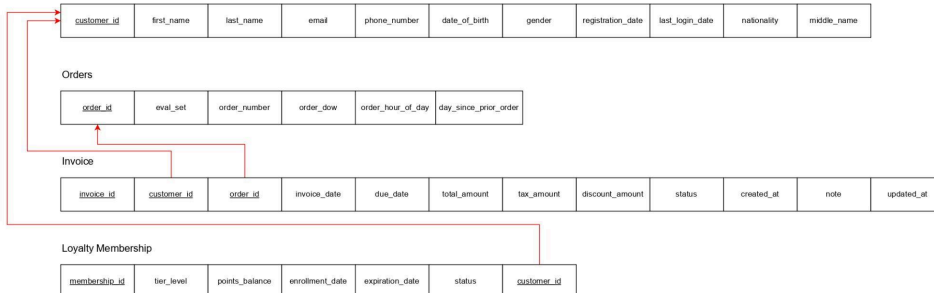
<u>order_id</u>	eval_set	order_number	order_dow	order_hour_of_day	day_since_prior_order
-----------------	----------	--------------	-----------	-------------------	-----------------------

Invoice

<u>invoice_id</u>	<u>customer_id</u>	<u>order_id</u>	invoice_date	due_date	total_amount	tax_amount	discount_amount	status	created_at	note	updated_at
-------------------	--------------------	-----------------	--------------	----------	--------------	------------	-----------------	--------	------------	------	------------

Loyalty Membership

<u>membership_id</u>	tier_level	points_balance	enrollment_date	expiration_date	status	<u>customer_id</u>
----------------------	------------	----------------	-----------------	-----------------	--------	--------------------



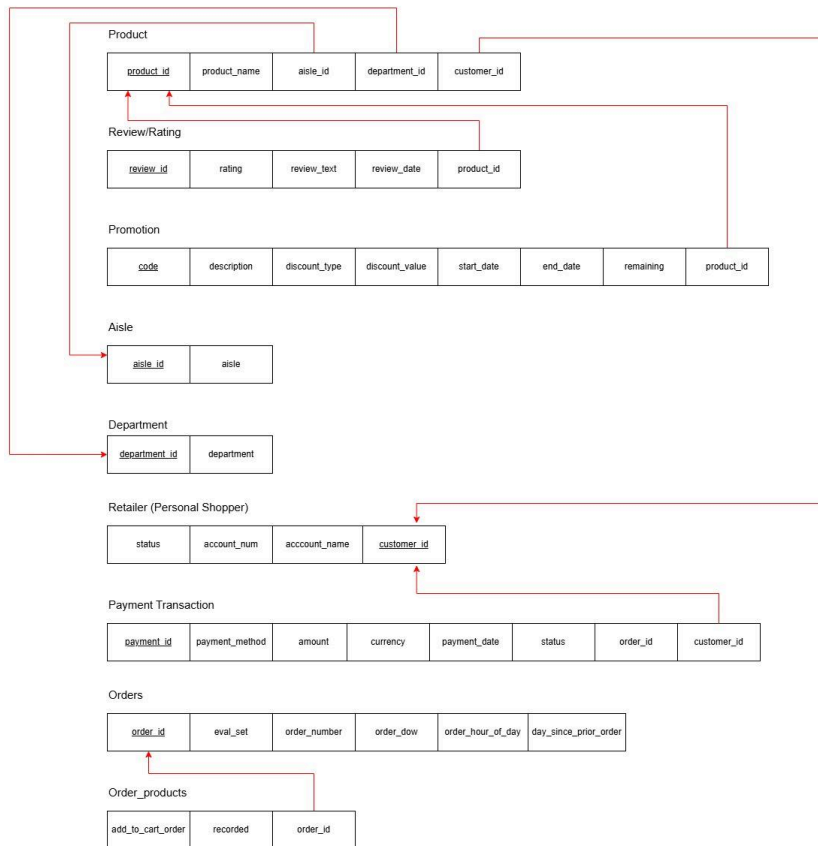
Step 3: Binary 1:1

Step 3 : 1-1



Step 4: Binary 1:M

Step 4 : 1-M



Step 5: Binary M:N

No Binary M:N Relation

Step 6: Multi-Valued

Step 6 : Multi-value

Customer

<u>customer_id</u>	first_name	last_name	email	phone_number	date_of_birth	gender	registration_date	last_login_date	nationality	middle_name
--------------------	------------	-----------	-------	--------------	---------------	--------	-------------------	-----------------	-------------	-------------

Address

street	city	province	postal_code	country	address_type	created_at	updated_at	house_number	customer_id	address_id
--------	------	----------	-------------	---------	--------------	------------	------------	--------------	-------------	------------

Step 7: N-ary

Step 7 : N-ary

Customer

<u>customer_id</u>	first_name	last_name	email	phone_number	date_of_birth	gender	registration_date	last_login_date	nationality	middle_name
--------------------	------------	-----------	-------	--------------	---------------	--------	-------------------	-----------------	-------------	-------------

Review/Rating

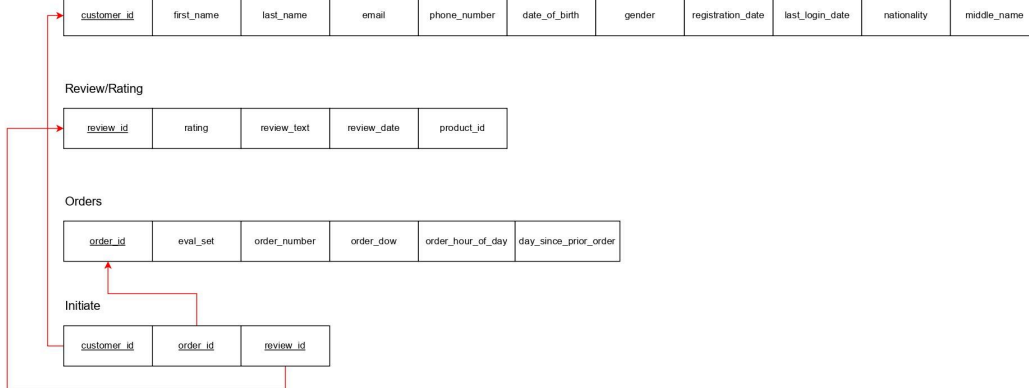
<u>review_id</u>	rating	review_text	review_date	product_id
------------------	--------	-------------	-------------	------------

Orders

<u>order_id</u>	eval_set	order_number	order_dow	order_hour_of_day	day_since_prior_order
-----------------	----------	--------------	-----------	-------------------	-----------------------

Initiate

<u>customer_id</u>	<u>order_id</u>	<u>review_id</u>
--------------------	-----------------	------------------



Step 8 (Option A): Specialization

Step 8 : Subtype

Loyalty Membership

<u>membership_id</u>	tier_level	points_balance	enrollment_date	expiration_date	status	<u>customer_id</u>
----------------------	------------	----------------	-----------------	-----------------	--------	--------------------

Bronze

<u>membership_id</u>	birthday_point
----------------------	----------------

Silver

<u>membership_id</u>	discount
----------------------	----------

Gold

<u>membership_id</u>	surprise_gift
----------------------	---------------

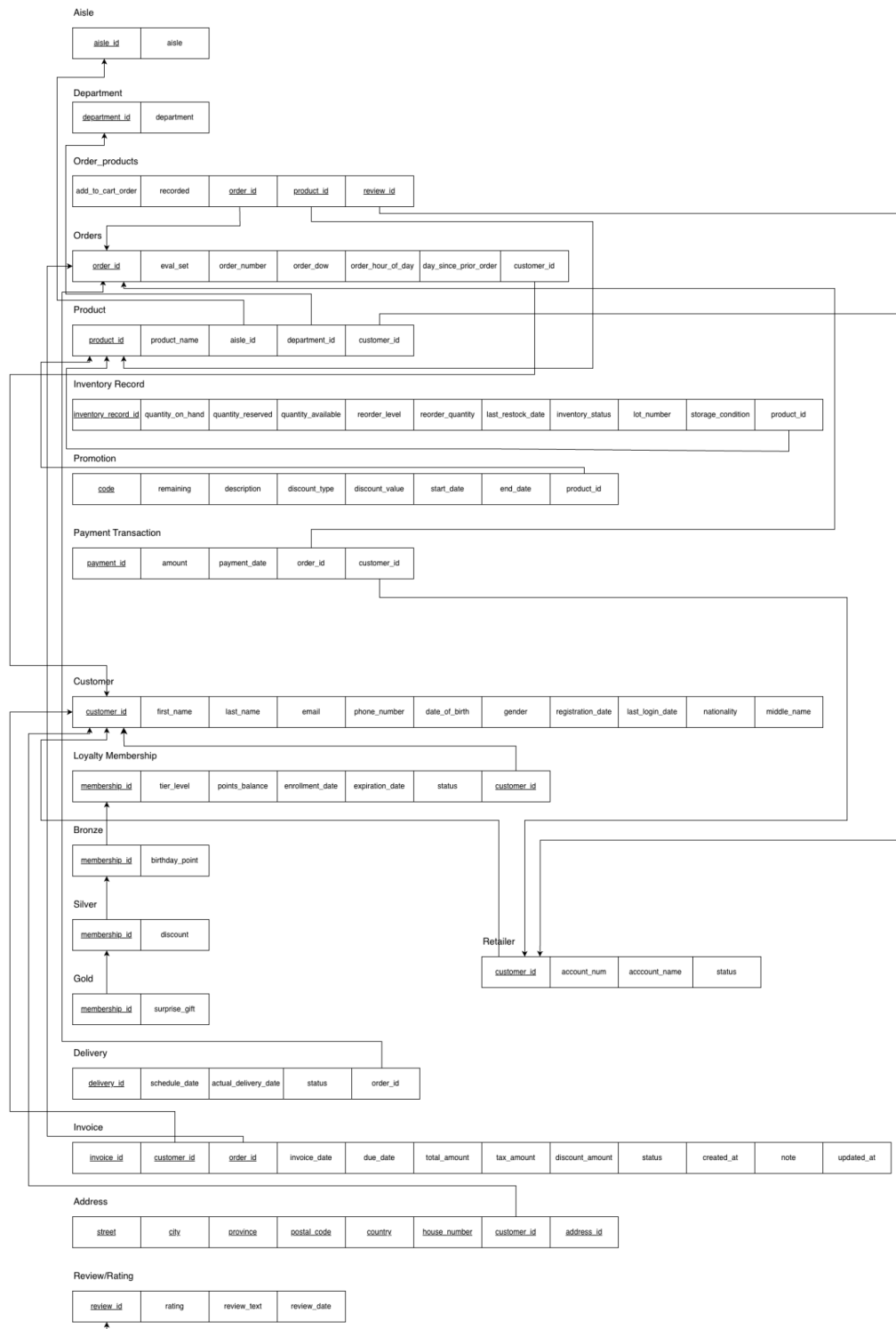
Customer

<u>customer_id</u>	first_name	last_name	email	phone_number	date_of_birth	gender	registration_date	last_login_date	nationality	middle_name
--------------------	------------	-----------	-------	--------------	---------------	--------	-------------------	-----------------	-------------	-------------

Retailer (Personal Shopper)

status	account_num	account_name	<u>customer_id</u>
--------	-------------	--------------	--------------------

Final relational diagram



Pre-Processing

The 3 files from Kaggle are too large to be implemented in our project, so we thought of a pre-processing method to scale down the files to only implement instances that are necessarily enough for each table. First off, we looked into all these three files: order, order_product (prior&train), and product; after a close inspection, we can scale down the file by relying on order table because it have two meaningful attributes (“order_hour_of_day” [0-23, representing the hour of the day] and “order_dow” [0-6, representing the date of the week]) that is fit to be used as a criteria for scaling down all the three files.

Remark:

Code needs to be run according to this order.

The code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

1. order_filter1:

Initially, we have about 3.5 million rows of instances in order tables from Kaggle. We then scaled it down to 1,000 rows of instances by depending on the two instances (“order_hour_of_day” and “order_dow”). First off, we set a grouping function for dividing the data according to both attributes: “order_dow” - 7 groups (1 group per day) and “order_hour_of_day” - 4 groups (each group has a range of 6 hours). We then do a permutation (pairing from the two groupings to be distinct groups): $7 \times 4 = 28$ groups.

Now, we randomly sampled data into the 28 groups: $1000/28 = 35$.

Code: `sampled = ( df.groupby([dow, hour_bin], group_keys=False) .apply(lambda x: x.sample(n=min(len(x), n_per_group), random_state=42)) )`

This code means that every instances are being implemented into each group according to a dataframe (“order_hour_of_day” and “order_dow”) and it also ensures that the instances that are sampled into the group will not exceed 35 rows, as that every group will have the same number of rows.

We then also random “user_id” for 200 customers from initially 1,000 instances. This is because 200 instances is sufficient enough for the table. Then we changed the column name from “user_id” to “customer_id”.

The end result will generate a new file called “orders_sampled”.

Code for order_filter1:

```
import pandas as pd
import numpy as np

# Load the CSV
df = pd.read_csv("D:/SIIT/Project_DB_Lecture/data/orders.csv")

# Define bins for order_hour_of_day
bins = [0, 6, 12, 18, 24]
labels = [0, 1, 2, 3] # temporary codes for 4 parts

# Temporary grouping
hour_bin = pd.cut(df["order_hour_of_day"], bins=bins, labels=labels, right=False)
dow = df["order_dow"]

# Number of groups and per-group sample size
n_total = 1000
n_groups = 7 * 4 # 28
n_per_group = n_total // n_groups # ~35

# Stratified sampling
sampled = (
    df.groupby([dow, hour_bin], group_keys=False)
    .apply(lambda x: x.sample(n=min(len(x), n_per_group), random_state=42))
)

# Reassign user_id randomly in range 1–200
sampled["user_id"] = np.random.randint(1, 201, size=len(sampled))

# Rename user_id → customer_id
sampled = sampled.rename(columns={"user_id": "customer_id"})
```

```
# Save only the original columns (with renamed column)
sampled = sampled.rename(columns={"user_id":
"customer_id"})[df.columns.str.replace("user_id", "customer_id")]
sampled.to_csv("orders_sampled.csv", index=False)

print(f'Created 'orders_sampled.csv' with {len(sampled)} rows, "
      f'customer_id reassigned randomly between 1 and 200.')

```

2. **order_product_filter2:**

We first concatenate "order_product_prior" and "order_product_train" into one file, "order_product_all". Then we will sample the order-product data by comparing and mapping the "order_id" from "order_product_all" to the "order_id" in "orders_sampled," saving the matched records as "order_products_filtered." Similarly, we will match the "order_id" from "orders_sampled" to those in "order_product_all" and save the resulting data as "orders_sampled_filtered."

Code for order_product_filter2:

```
import pandas as pd

# Load files
orders_sampled = pd.read_csv("D:/SIIT/Project_DB_Lecture/orders_sampled.csv")
op_prior = pd.read_csv("D:/SIIT/Project_DB_Lecture/data/order_products__prior.csv")
op_train = pd.read_csv("D:/SIIT/Project_DB_Lecture/data/order_products__train.csv")

# Step 1: combine prior + train
order_products_all = pd.concat([op_prior, op_train], ignore_index=True)

# Step 2: filter combined order_products by sampled order_ids
order_products_filtered = order_products_all[
    order_products_all["order_id"].isin(orders_sampled["order_id"])
]

# Step 3: filter orders_sampled to keep only those present in combined file
orders_sampled_filtered = orders_sampled[
    orders_sampled["order_id"].isin(order_products_filtered["order_id"])
]

```

```
# Save results
order_products_filtered.to_csv("order_products_filtered.csv", index=False)
orders_sampled_filtered.to_csv("orders_sampled_filtered.csv", index=False)

print(f'Kept {len(order_products_filtered)} order-product rows "
      f"and {len(orders_sampled_filtered)} sampled orders.")
```

3. **order_product_review3:**

We first extract unique order_id from “order_products_filtered”, then create a lookup table that maps each unique order_id to a sequential review number starting at 1 (Order_id is being mapped to a sequential review_id, which is based on the order of appearance [index-like numbering] of unique order_ids), and store the mapping in a dictionary. Then we assigned each order_id a corresponding review_id based on the mapping of order_id and review_id stored in the dictionary. Lastly, we saved the new file as “order_products_with_review”

Code for order_product_review3:

```
import pandas as pd

# Load your combined order_products file
df = pd.read_csv("D:/SIIT/Project_DB_Lecture/order_products_filtered.csv")

# Step 1: create a mapping from order_id → sequential review_id
unique_orders = df["order_id"].drop_duplicates().reset_index(drop=True)
order_to_review = {order_id: idx+1 for idx, order_id in enumerate(unique_orders)}

# Step 2: assign review_id based on order_id
df["review_id"] = df["order_id"].map(order_to_review)

# Step 3: reorder columns so review_id is between product_id and add_to_cart_order
cols = list(df.columns)
# find positions
pid_idx = cols.index("product_id")
# rebuild column order
new_cols = cols[:pid_idx+1] + ["review_id"] + cols[pid_idx+1:-1] + [cols[-1]]
df = df[new_cols]
```

```
# Save result
df.to_csv("order_products_with_review.csv", index=False)

print("Added review_id column between product_id and add_to_cart_order.")
```

4. **order_order_product_mapped4:**

Here we want to solve the order_id with irregular spacing, jumping of numbers (for example. order_id: 1, 500, 2000), in “order_products_with_review” and “orders_sampled_filtered”. We then create a set that maps old order_id to a new sequential order_id, starting from 1. Afterwards, we map the old order_id in both files to the set we created and overwrite it. Lastly, we saved “order_products_with_review” in the new file as “order_product_with_review_mapped” and “orders_sampled_filtered” as “orders_sampled_filtered_mapped”.

Code for order_order_product_mapped4:

```
import pandas as pd

# Load both files
op = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review.csv")
orders = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/orders_sampled_filtered.csv")

# Collect all unique order_ids across both files
all_order_ids = pd.concat([op["order_id"],
orders["order_id"]]).drop_duplicates().reset_index(drop=True)

# Build mapping: old order_id → new continuous id
order_id_map = {old_id: new_id for new_id, old_id in enumerate(all_order_ids, start=1)}

# Overwrite the original order_id in both DataFrames
op["order_id"] = op["order_id"].map(order_id_map)
orders["order_id"] = orders["order_id"].map(order_id_map)

# Save results (overwriting old order_id with mapped values)
op.to_csv("order_products_with_review_mapped.csv", index=False)
orders.to_csv("orders_sampled_filtered_mapped.csv", index=False)

print("Replaced order_id with continuous values starting from 1 in both files.")
```

5. **products_filter5:**

Here we want to scale down the data of the products table from 50,000 instances. Foremost, we load the “product” and the “order_products_with_review_mapped” file; then compare the product_id from “product” to “order_products_with_review_mapped” and use only the one with the same product_id, which resulted in 4,500 instances. This means that these products are all products that have been ordered before. However, we want to also include and sample products that have not been ordered before, so we use the “not function” to the code we used to map product_id from “product” to “order_products_with_review_mapped” previously.

Code: for product that have not been ordered before:

```
remaining_products = products[~products["product_id"].isin(filtered_products["product_id"])]
```

This code will result in about 45,500 instances from the original “product” table. Then, we randomly sampled the data out to only 500 instances and concatenated it to the 4,500 instances we sampled before this and saved the new file as “products_filtered_plus_500”. We only sampled out 500 instances from ‘product that have not been ordered before’, because it is sufficient enough for describing the table after concatenating with the 4,500 ‘products that have been ordered before’ instances.

Code for products_filter5:

```
import pandas as pd
```

```
# Load files
```

```
products = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/data/products.csv")
```

```
order_products =
```

```
pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped.csv")
```

```
# Step 1: filter products to only those used in order_products
```

```
filtered_products = products[products["product_id"].isin(order_products["product_id"])]
```

```
# Step 2: find products not already in filtered_products set
```

```
remaining_products = products[~products["product_id"].isin(filtered_products["product_id"])]
```

```
# Step 3: randomly select 500 from remaining (adjust if fewer than 500 available)
```

```
extra_products = remaining_products.sample(n=500, random_state=42)
```

```
# Step 4: combine filtered + extra (500)
```

```
final_products = pd.concat([filtered_products, extra_products], ignore_index=True)
```

```
# Step 5: save result
final_products.to_csv("products_filtered_plus500.csv", index=False)

print(f'Created products_filtered_plus500.csv with {len(final_products)} rows "
      f'({len(filtered_products)} filtered + 500 extra unique).')
```

6. **product_order_product_map6:**

Here we want to solve the order_id with irregular spacing, jumping of numbers (for example. product_id: 1, 500, 2000), in “products_filtered_plus500” and “products_with_review_mapped”. We then create a set that maps old product_id to a new sequential product_id, starting from 1. Afterwards, we map the old product_id in both files to the set we created and overwrite it. Lastly, we saved “products_filtered_plus500” in the new file as “products_filtered_plus500_mapped” and “products_with_review_mapped” as “products_with_review_mapped2”.

Code for product_order_product_map6:

```
import pandas as pd

# Load both files
products = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/products_filtered_plus500.csv")
order_products = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped.csv")

# Collect all unique product_ids across both files
all_product_ids = pd.concat([products["product_id"], order_products["product_id"]]) \
    .drop_duplicates().reset_index(drop=True)

# Build mapping: old product_id → new continuous id
product_id_map = {old_id: new_id for new_id, old_id in enumerate(all_product_ids, start=1)}

# Overwrite product_id in both DataFrames
products["product_id"] = products["product_id"].map(product_id_map)
order_products["product_id"] = order_products["product_id"].map(product_id_map)
```

```
# Save results
products.to_csv("products_filtered_plus500_mapped.csv", index=False)
order_products.to_csv("order_products_with_review_mapped2.csv", index=False)

print("Replaced product_id with continuous values starting from 1 in both files.")
```

7. **product_with_retailer7:**

First, we loaded the file “products_filtered_plus500_mapped”, which contains the scaled-down and re-mapped product_ids. We then counted the number of rows (n) to know how many retailer_ids we need to generate.

First off, we need to understand that the customer_table has a total of 300 instances and 150 of the instances are ‘buyer-only’, which lies in the range of 1-200 instances; while the other 50 instances are both buyer and retailer. Furthermore, the instances from 201-300 are retailer only.

We then randomly selected 50 distinct numbers from the range 1–200. This number is used as a threshold combining customer (buyer only) and customer that is both the buyer and retailer.

Code:

```
prob = 0.3
mask = np.random.rand(n) < prob
```

This code created a probabilistic assignment:

- With 30% probability, a product is assigned to a retailer from the chosen set of 50 numbers.
- With 70% probability, a product is assigned a retailer_id randomly drawn from the range 201–300.

Code:

```
retailer_id = np.empty(n, dtype=int)
retailer_id[mask] = np.random.choice(chosen_numbers, size=mask.sum(), replace=True)
retailer_id[~mask] = np.random.randint(201, 301, size=(~mask).sum())
```

From this code, we generated a boolean mask of length n where each entry is True with probability (0.3). Products corresponding to True entries get their retailer from the chosen set, while the rest get their retailer from the 201–300 range.

We then added this new retailer_id column to the products_table’s DataFrame and saved the result as “products_filtered_plus500_mapped_with_retailer”.

Code for product_with_retailer7:

```

import pandas as pd
import numpy as np

# Load the file
Products
= pd.read_csv(r"D:/SIIT/Project_DB_Lecture/products_filtered_plus500_mapped.csv")

n = len(products)

# Step 1: randomly select 50 distinct numbers from 1–200
chosen_numbers = np.random.choice(range(1, 201), size=50, replace=False)

# Step 2: assign retailer_id
# For each row, flip a coin: either pick from chosen_numbers or from 201–300
# Example: 30% chance from chosen_numbers, 70% from 201–300
prob = 0.3
mask = np.random.rand(n) < prob

retailer_id = np.empty(n, dtype=int)
retailer_id[mask] = np.random.choice(chosen_numbers, size=mask.sum(), replace=True)
retailer_id[~mask] = np.random.randint(201, 301, size=(~mask).sum())

# Step 3: add column
products["retailer_id"] = retailer_id

# Save result

products.to_csv("products_filtered_plus500_mapped_with_retailer.csv", index=False)

print(f"Added retailer_id column: using 50 chosen numbers from 1–200 (reused across rows), "
      f"and 201–300 for the rest.")

```

8. **order_product_final8:**

We load the “order_products_with_review_mapped2” file. From the file, the code accidentally generates an extra column named “review_id.1”, so we just simply delete it and save the new file as “order_products_with_review_mapped_final”.

Code for order_product_final8:

```
import pandas as pd

# Load the file
df = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped2.csv")

# Drop the unwanted column
if "review_id.1" in df.columns:
    df = df.drop(columns=["review_id.1"])

# Save back to the same file (or a new one if you prefer)
df.to_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped_final.csv",
index=False)

print("Column 'review_id.1' has been removed.")
```

Data Dictionary

1. Aisles.csv (Master)

- **aisle_id** : This is the unique identifier for each aisle. This numerical ID is used to link products to their specific aisle.
- **aisle** : This is the name of the aisle or category within the store in a human-readable format, such as "frozen juice," "cereal," or "dry pasta."

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Aisles	aisle_id	This is the unique identifier for each aisle. Used to link products to their specific aisle.	Int unsigned	x	No		PK	
	aisle	The name of the aisle or category in a human-readable format (e.g., "frozen juice", "cereal", etc.).	varchar (50)	xxxxx Text (Lower case)	No			

2. Department.csv (Master)

- **department_id** : The unique identifier for each department. This numerical ID is used to link products and aisles to their specific department.
- **department** : The name of the major section or category where related products are grouped together, such as "bakery", "alcohol", or "breakfast".

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Department	department_id	The unique identifier for each department. This numerical ID links products (and possibly aisles) to departments.	Integer	x	No		PK	
	department	The name of the major section or product category, such as "bakery", "alcohol", or "breakfast".	Varchar(20)	xxxxx Text (Lowercase)	No			

3. Order_products.csv (Many to Many Relationship)

- **order_id** : A unique identifier for the order.
- **product_id** : A unique identifier for the product.
- **add_to_cart_order** : A number that shows the order in which the product was added to the customer's cart.
- **reordered** : A binary flag (1 or 0) that shows if the customer has purchased this product before. 1 means they have, and 0 means this is the first time.

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Order_products	order_id	A unique identifier for the order. Links to the orders.csv file.	int unsigned	x	No		PK, FK	order_id [Orders]
	product_id	A unique identifier for the product. Links to the products.csv file.	int unsigned	x	No		PK, FK	product_id [Product]
	add_to_cart_order	Indicates the sequence in which the product was added to the customer's cart during this order.	Int unsigned	x	No			
	reordered	A binary flag: 1 if the product was ordered before by the customer, 0	bit	x	No	0, 1		
	review_id	Unique review identifier.	int unsigned	x	No		PK, FK	review_id [Review/Rating]

4. Orders.csv (Transaction)

- **order_id** : A unique ID number for each order.
- **eval_set** : To divide customer order data into three main groups for training and testing a machine learning model. Each group has the following meaning:

prior: Consists of **all past orders** from the majority of users. This data is used as a foundation to "teach" the model to learn each user's purchasing behavior.

train: Consists of the **single most recent order** from a group of users used to train the model. This data serves as the "correct answer" for the model to compare against and improve its prediction accuracy.

test: Consists of the **single most recent order** from another group of users. This data does not contain the actual products purchased (which is the data we want to predict) and is used to evaluate how accurately the model can predict.

- **order_number** : The chronological number of the order for a specific user (e.g., their 1st, 2nd, or 3rd order).
- **order_dow** : The day of the week the order was placed. Usually **0** for Sunday, **1** for Monday, etc.
- **order_hour_of_day** : The hour of the day the order was placed (in a 24-hour format).
- **day_since_prior_order** : The number of days since the user's last order. This is a missing value for a user's first order.

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Orders	order_id	A unique ID number for each order.	int unsigned	x	No		PK	
	eval_set	Indicates how the order is used in modeling: • prior: all historical orders for training. • train: latest known order	enum ('prior', 'train', 'test')	xxxxxx	No			

		used to compare predictions. • test: latest order for which products are to be predicted.						
	order_number	The sequential number of the order for the specific user (e.g., their 1st, 2nd, or 3rd order).	int unsigned	x	No			
	order_dow	The day of the week the order was placed. Usually 0 for Sunday, 1 for Monday, etc.	int	x	No	0 to 6		
	order_hour_of_day	The hour of the day the order was placed (in a 24-hour format).	int	x	No	0 to 23		
	day_since_prior_order	The number of days since the user's last order. This is a missing value for a user's first order.	float	12.3	Yes			
	customer_id	Unique customer identifier.	int unsigned	x	No		FK	customer_id [Customer]

5. Product.csv (Master)

- **product_id** : A unique number that identifies each product.
- **product_name** : The name of the product, like "Organic Turkey Burgur" or "White Pearl Onions."
- **aisle_id** : The ID number of the specific aisle where the product is located.
- **department_id** : The ID number of the department the product belongs to.

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Product	product_id	A unique number that identifies each product.	int unsigned	x	No		PK	
	product_name	The name of the product, such as "Organic Turkey Burger" or "White Pearl Onions."	varchar(150)	Xxxxx	No			
	aisle_id	The ID of the aisle where the product is located. Links to aisles.csv.	int unsigned	x	No		FK	aisle_id [Aisle]
	department_id	The ID of the department the product belongs to. Links to departments.csv.	int unsigned	x	No		FK	department_id [Department]
	retailer_ID	A unique ID number for each customer who placed the order. Links to the customer.csv file.	int unsigned	x	No		FK	customer_id [Retailer]

6. Promotion (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Promotion	code	Coupon code string.	varchar(20)	xxxxx	No		PK	
	remaining	Remaining of coupons	int unsigned	x	Yes			
	description	Details of the promotion.	varchar(255)	Xxxxxx	Yes			
	discount_type	Type (percentage, fixed_amount).	enum ('Percentage', 'Fixed_amount')	xxxxx	No			
	discount_value	Discount value.	float	x.xx	No			
	start_date	Valid-from date.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	end_date	Expiration date.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	product_id	A unique identifier for the product. Links to the products.csv file.	int unsigned	x	No		FK	product_id [Product]

7. Payment Transaction (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Payment Transaction	payment_id	Unique payment transaction ID.	int unsigned	x	No		PK	
	order_id	Associated order identifier. Links to the orders.csv file.	int unsigned	x	No		FK	order_id [Orders]
	amount	Amount paid.	float	x.xx	No			
	payment_date	When payment was made.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	customer_id	A unique ID number for each customer who placed the order. Links to the customer.csv file.	int unsigned	x	No		FK	retailer_id [retailer]

8. Invoice (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Invoice	invoice_id	Unique invoice document ID.	int unsigned	x	No		PK	
	order_id	Related order identifier. Links to the orders.csv file.	int unsigned	x	No		PK, FK	order_id [Orders]
	invoice_date	When invoice was generated.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	due_date	Payment due date.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	total_amount	Total billed amount.	float	x.xx	No			
	tax_amount	Tax portion of invoice.	float	x.xx	No			
	discount_amount	Discounts applied.	float	x.xx	Yes			
	customer_id	A unique ID number for each customer who placed the order. Links to the customer.csv file.	int unsigned	x	No		PK, FK	customer_id [Customer]
	status	Invoice status (draft, sent, paid).	Enum	Enum('Paid', 'Unpaid', 'Overdue')	No			

	created_at	When the invoice record was created.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	note	Extra note	varchar(255)	xxxxx	Yes			
	updated_at	Last update timestamp.	datetime	YYYY-MM-DDTHH:MM:SS	No			

9. Customer (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Customer	customer_id	Unique customer identifier.	Int unsigned	x	No		PK	
	first_name	Customer's given name.	varchar(50)	Xxxxx	No			
	middle_name	middle name of customer	varchar(50)	Xxxxx	Yes		No	
	last_name	Customer's family name.	varchar(50)	Xxxxx	No		No	
	email	Primary email for contact.	varchar(100)	xxxxx	No		Unique	
	phone_number	Primary phone number.	varchar(15)	xxxxx	No			
	date_of_birth	Customer's birth date.	date	YYYY-MM-DD	No			
	gender	Gender identity.	enum ('Male', 'Female', 'Other', 'Preferred not to say')	Xxxxx	No			
	registration_date	When a customer registered.	datetime	YYYY-MM-DD THH:MM:SS	No			
	last_login_date	Last login timestamp.	datetime	YYYY-MM-DD THH:MM:SS	Yes			

	nationality	nationality of customer	varchar(50)	Xxxxx	No			
--	--------------------	-------------------------	-------------	-------	----	--	--	--

10. Retailer - subtype of Customer (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Customer (Retailer)	status	Employment status (active, inactive).	enum('active','inactive')	xxxxx	No			
	account_number	The legal number for the bank account of a retailer.	vachar(20)	xxxx	No			
	account_name	The legal name for the bank account of a retailer.	vachar(100)	Xxxxx	No			
	retailer_id	Unique customer identifier.	int unsigned	x	No		PK, FK	customer_id [Customer]

11. Loyalty Membership (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership	membership_id	Unique loyalty membership ID.	int	x	No		PK	
	customer_id	Linked customer.	int	x	No		PK, FK	customer_id [Customer]
	tier_level	Membership tier (bronze, silver, gold).	enum('bronze', 'silver', 'gold')	xxxxx	No			
	points_balance	Current points balance.	Int unsigned	x	No			
	enrollment_date	When a customer enrolled.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	expiration_date	Program expiration date.	datetime	YYYY-MM-DDTHH:MM:SS	Yes			
	status	Membership status (active, expired).	enum('active', 'expired')	xxxxx	No			

12. Loyalty Membership (Bronze) - subtype of Loyalty Membership (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership (Bronze)	membership_id	Unique loyalty membership ID.	int unsigned	x	No		PK, FK	membership_id[Loyalty Membership]
	birthday_point	Number of loyalty points automatically credited to a member on their birthday anniversary each year	int unsigned	x	No		No	

13. Loyalty Membership (Silver) - subtype of Bronze (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership (Silver)	membership_id	Unique loyalty membership ID.	int unsigned	x	No		PK, FK	membership_id[Loyalty Membership (Bronze)]
	Discount_percent	Percentage discount on eligible purchases, determined by the member's tier	int unsigned	x	No		No	

14. Loyalty Membership (Gold) - subtype of Sliver (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership (Gold)	membership_id	Unique loyalty membership ID.	int unsigned	x	No		PK, FK	membership_id[Loyalty Membership (Silver)]
	surprise_gift	Random member reward granted without notice	Enum (“airplane ticket”, “promo code”, “gift card”)	xxxxx	No		No	

15. Delivery (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Delivery	delivery_id	Unique delivery event ID.	int unsigned	x	No		PK	
	order_id	Order being delivered. Links to the orders.csv file.	int unsigned	x	No		FK	order_id [Orders]
	scheduled_date	When delivery is scheduled.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	actual_delivery_date	When delivery actually occurred.	datetime	YYYY-MM-DDTHH:MM:SS	Yes			
	status	Delivery status (pending, in_transit, done)	enum('Pending', 'Shipped', 'Delivered', 'Cancelled')	Xxxxxx	No			

16. Inventory Record (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Inventory Record	inventory_record_id	Unique inventory record ID.	int unsigned	x	No		PK	
	product_id	Product in stock.	int unsigned	x	No		FK	Product_id [Product]
	quantity_on_hand	Units physically in stock.	int unsigned	x	No			
	quantity_reserved	Units reserved (e.g., carts, orders).	int unsigned	x	No			
	quantity_available	Units available for sale.	int unsigned	x	No			
	reorder_level	Threshold to trigger restock.	int unsigned	x	No			
	reorder_quantity	Default restock quantity.	int unsigned	x	No			
	last_restock_date	When the product was last restocked.	datetime	YYYY-MM-DDTHH:MM:SS	Yes			
	inventory_status	Status (in_stock, out_of_stock, backordered)	Enum (In Stock, Out of Stock, backordered)	Xxxxx	No			

	lot_number	Supplier lot or batch number for traceability.	varchar(50)	xxxxxxxxx	Yes			
	storage_condition	Condition of where to store the product	varchar(100)	Xxxxx	Yes			

17. Review / Rating (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Review / Rating	review_id	Unique review identifier.	int unsigned	x	No		PK	
	rating	Score from 1 to 5.	int	x	No	1-5		
	review_text	Optional text feedback.	varchar(255)	Xxxxxx	Yes			
	review_date	When a review was submitted.	datetime	YYYY-MM-DDTHH:MM:SS	No			

18. Address (Multivalued and Composite Attribute)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Address	address_id	Unique identifier for each address.	int unsigned (Multivalued attribute)	x	No		PK	
	street	Street number and name.	varchar(50)	Xxxxx	No		PK	
	city	City name.	varchar(50)	Xxxxx	No		PK	
	province	Province or state.	varchar(50)	Xxxxx	No		PK	
	postal_code	Postal or ZIP code.	varchar(10)	Xxxxx	No		PK	
	country	Country name.	varchar(50)	Xxxxx	No		PK	
	house_number	House's number	varchar(20)	xxxxx	No		PK	
	customer_id	Unique customer identifier.	int unsigned	x	No		PK, FK	customer_id [Customer]

Create a database in MAMP Server (DDL)

Based on the data dictionary

-- This SQL script creates a database named 'DB_Lecture_Project' if it does not already exist, and then sets it as the current database for subsequent operations.

```
CREATE DATABASE IF NOT EXISTS DB_Lecture_Project;
USE DB_Lecture_Project;
```

-- 1st table creations: “aisles”

```
CREATE TABLE IF NOT EXISTS aisles (
    aisle_ID INT unsigned not null AUTO_INCREMENT,
    aisle VARCHAR(50) NOT NULL,
    constraint PK_aisles primary key (aisle_ID)
);
```

-- 2nd table creations: “departments”

```
CREATE TABLE IF NOT EXISTS departments (
    department_ID INT unsigned not null AUTO_INCREMENT,
    department VARCHAR(20) NOT NULL,
    constraint PK_departments primary key (department_ID)
);
```

-- 3rd table creations: “customer”

```
CREATE TABLE IF NOT EXISTS customer (
    customer_ID INT unsigned not null AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    middle_name VARCHAR(50),
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender ENUM('Male', 'Female', 'Other', 'Preferred not to say') NOT NULL,
    registration_date DATETIME NOT NULL,
    last_login_date DATETIME,
    nationality VARCHAR(50) NOT NULL,
    constraint PK_customers primary key (customer_ID),
    constraint UQ_customer_email UNIQUE (email)
);
```

-- 4th table creations: “loyalty_membership”

```
CREATE TABLE IF NOT EXISTS loyalty_membership (
  membership_ID INT unsigned not null AUTO_INCREMENT,
  customer_ID INT unsigned not null,
  tier_level ENUM('Bronze', 'Silver', 'Gold') NOT NULL,
  points_balance INT unsigned NOT NULL,
  enrollment_date DATETIME NOT NULL default CURRENT_TIMESTAMP,
  expiration_date DATETIME,
  status ENUM('Active', 'Expired') NOT NULL,
  primary key (membership_ID, customer_ID),
  CONSTRAINT FK_loyalty_membership_customer FOREIGN KEY (customer_ID)
REFERENCES customer(customer_ID)
);
```

-- 5th table creations: “bronze”

```
CREATE TABLE IF NOT EXISTS bronze (
  membership_ID INT unsigned not null,
  birthday_point INT unsigned not null,
  constraint PK_bronze primary key (membership_ID),
  CONSTRAINT FK_bronze_loyalty_membership FOREIGN KEY (membership_ID)
REFERENCES loyalty_membership(membership_ID)
);
```

-- 6th table creations: “silver”

```
CREATE table if not exists silver (
  membership_ID INT unsigned not null,
  discount_percent INT unsigned not null,
  constraint PK_silver primary key (membership_ID),
  CONSTRAINT FK_silver_bronze FOREIGN KEY (membership_ID) REFERENCES
bronze(membership_ID)
);
```

-- 7th table creations: “gold”

```
CREATE TABLE IF NOT EXISTS gold (
  membership_ID INT unsigned not null,
  surprise_gift ENUM('Airplane Ticket', 'Gift Card', 'Promotion Code') NOT NULL,
  constraint PK_gold primary key (membership_ID),
  CONSTRAINT FK_gold_silver FOREIGN KEY (membership_ID) REFERENCES
silver(membership_ID)
);
```

-- 8th table creations: “retailer”

```
CREATE TABLE IF NOT EXISTS retailer (
  retailer_ID INT unsigned not null,
  status ENUM('Active', 'Inactive') NOT NULL,
  account_number VARCHAR(20) NOT NULL,
  account_name VARCHAR(100) NOT NULL,
  constraint PK_retailer primary key (retailer_ID),
  CONSTRAINT FK_retailer_customer FOREIGN KEY (retailer_ID) REFERENCES
customer(customer_ID)
);
```

-- 9th table creations: “review”

```
CREATE TABLE IF NOT EXISTS review (
  review_ID INT unsigned not null AUTO_INCREMENT,
  rating INT NOT NULL,
  review_text VARCHAR(255),
  review_date DATETIME NOT NULL default CURRENT_TIMESTAMP,
  constraint PK_reviews primary key (review_ID),
  CONSTRAINT chk_rating_range CHECK (rating BETWEEN 1 AND 5)
);
```

-- 10th table creations: “Orders”

```
CREATE TABLE IF NOT EXISTS orders (
  order_ID INT unsigned not null AUTO_INCREMENT,
  customer_ID INT unsigned not null,
  eval_set ENUM('train','prior','test') NOT NULL,
  order_number INT unsigned NOT NULL,
  order_dow INT NOT NULL,
  order_hour_of_day INT NOT NULL,
  days_since_prior_order Float,
  constraint PK_orders primary key (order_ID),
  CONSTRAINT FK_orders_customer FOREIGN KEY (customer_ID) REFERENCES
customer(customer_ID),
  CONSTRAINT chk_order_dow_range CHECK (order_dow BETWEEN 0 AND 6),
  CONSTRAINT chk_order_hour_of_day_range CHECK (order_hour_of_day BETWEEN 0
AND 23)
);
```

-- 11th table creations: “products”

```
CREATE TABLE IF NOT EXISTS products (
  product_ID INT unsigned not null AUTO_INCREMENT,
  product_name VARCHAR(150) NOT NULL,
  aisle_ID INT unsigned NOT NULL,
  department_ID INT unsigned NOT NULL,
  retailer_ID INT unsigned NOT NULL,
  constraint PK_products primary key (product_ID),
  CONSTRAINT FK_products_aisles FOREIGN KEY (aisle_ID) REFERENCES
aisles(aisle_ID),
  CONSTRAINT FK_products_departments FOREIGN KEY (department_ID) REFERENCES
departments(department_ID),
  CONSTRAINT FK_products_retailer FOREIGN KEY (retailer_ID) REFERENCES
retailer(retailer_ID)
);
```

-- 12th table creations: “order_products”

```
CREATE TABLE IF NOT EXISTS order_products (
  order_ID INT unsigned not null,
  product_ID INT unsigned not null,
  review_ID INT unsigned not null,
  add_to_cart_order INT unsigned NOT NULL,
  reordered INT NOT NULL,
  primary key (order_ID, product_ID, review_ID),
  constraint FK_order_products_review FOREIGN KEY (review_ID) REFERENCES
review(review_ID),
  CONSTRAINT FK_order_products_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID),
  CONSTRAINT FK_order_products_products FOREIGN KEY (product_ID) REFERENCES
products(product_ID)
);
```

-- 13th table creations: “inventory_record”

```
CREATE TABLE IF NOT EXISTS inventory_record (
  inventory_ID INT unsigned not null AUTO_INCREMENT,
  product_ID INT unsigned not null,
  quantity_on_hand INT unsigned NOT NULL,
  quantity_reserved INT unsigned NOT NULL,
  quantity_available INT unsigned NOT NULL,
  reorder_level INT unsigned NOT NULL,
  reorder_quantity INT unsigned NOT NULL,
  last_restock_date DATETIME,
  inventory_status ENUM('In Stock', 'Out of Stock', 'backordered') NOT NULL,
  lot_number VARCHAR(50),
  storage_conditions VARCHAR(100),
  constraint PK_inventory_records primary key (inventory_ID),
  CONSTRAINT FK_inventory_records_products FOREIGN KEY (product_ID)
REFERENCES products(product_ID)
);
```

-- 14th table creations: “promotion”

```
CREATE TABLE IF NOT EXISTS promotion (
  code VARCHAR(20) not null,
  product_ID INT unsigned not null,
  remaining INT unsigned,
  description VARCHAR(255),
  discount_type ENUM('Percentage', 'Fixed Amount') NOT NULL,
  discount_value Float NOT NULL,
  start_date DATETIME NOT NULL,
  end_date DATETIME NOT NULL,
  constraint PK_promotion primary key (code),
  CONSTRAINT FK_promotion_products FOREIGN KEY (product_ID) REFERENCES
products(product_ID)
);
```

-- 15th table creations: “payment_transaction”

```
CREATE TABLE IF NOT EXISTS payment_transaction (
  payment_ID INT unsigned not null AUTO_INCREMENT,
  order_ID INT unsigned not null,
  customer_ID INT unsigned not null,
  payment_method ENUM('Credit Card', 'Debit Card', 'PayPal', 'Bank Transfer') NOT NULL,
  amount Float NOT NULL,
  currency VARCHAR(10) NOT NULL,
  payment_date DATETIME NOT NULL,
  status ENUM('Successful', 'Failed', 'Pending') NOT NULL,
  constraint PK_payment_transaction primary key (payment_ID),
  CONSTRAINT FK_payment_transaction_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID),
  CONSTRAINT FK_payment_transaction_retailer FOREIGN KEY (customer_ID)
REFERENCES retailer(retailer_ID)
);
```

-- 16th table creations: “invoice”

```

CREATE TABLE IF NOT EXISTS invoice (
    invoice_ID INT unsigned not null AUTO_INCREMENT,
    order_ID INT unsigned not null,
    customer_ID INT unsigned not null,
    invoice_date DATETIME NOT NULL default CURRENT_TIMESTAMP,
    due_date DATETIME NOT NULL,
    total_amount Float NOT NULL,
    tax_amount Float NOT NULL,
    discount_amount Float,
    status ENUM('Paid', 'Unpaid', 'Overdue') NOT NULL,
    created_at DATETIME NOT NULL default CURRENT_TIMESTAMP,
    updated_at DATETIME,
    note VARCHAR(255),
    primary key (invoice_ID, order_ID, customer_ID),
    CONSTRAINT FK_invoice_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID),
    CONSTRAINT FK_invoice_customer FOREIGN KEY (customer_ID) REFERENCES
customer(customer_ID)
);

```

-- 17th table creations: “delivery”

```

CREATE TABLE IF NOT EXISTS delivery (
    delivery_ID INT unsigned not null AUTO_INCREMENT,
    order_ID INT unsigned not null,
    scheduled_date DATETIME NOT NULL,
    actual_delivery_date DATETIME,
    status ENUM('Pending', 'Shipped', 'Delivered', 'Cancelled') NOT NULL,
    constraint PK_delivery primary key (delivery_ID),
    CONSTRAINT FK_delivery_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID)
);

```

-- 18th table creations: "address"

```
CREATE TABLE IF NOT EXISTS address (  
    address_ID INT unsigned not null AUTO_INCREMENT,  
    customer_ID INT unsigned not null,  
    house_number VARCHAR(20) NOT NULL,  
    street VARCHAR(50) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    province VARCHAR(50) NOT NULL,  
    postal_code VARCHAR(10) NOT NULL,  
    country VARCHAR(50) NOT NULL,  
    primary key (address_ID, customer_ID, house_number, street, city, province, postal_code,  
country),  
    CONSTRAINT FK_address_customer FOREIGN KEY (customer_ID) REFERENCES  
customer(customer_ID)  
);
```

Example of Data Manipulation Language (DML)

1. #parser_table1

We first load the aisle data by reading the file aisles.csv from the 'files location' directory and inserting its records into the aisles table, skipping the first header row. Similarly, we will load the department data from departments.csv in the same directory and insert those records into the departments table, also skipping its header row.

Code:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/aisles.csv'

insert aisles.csv data into aisles table

```
LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_sampled_2/aisles.csv'
INTO TABLE aisles
CHARACTER SET 'utf8mb4'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/departments.csv'

insert departments.csv data into departments table

```
LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_sampled_2/departments.csv'
INTO TABLE departments
CHARACTER SET 'utf8mb4'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

2. #parser_table2

We insert records into “customer” table, specifying which contains ‘first_name’, ‘last_name’, ‘email’, ‘phone_number’, ‘date_of_birth’, and ‘nationality’. Similarly to the customer, we will insert records into “bronze” table, providing the ‘membership_id’ and its corresponding to ‘birthday_point’ for each. For “silver” table, saving records that map a ‘membership_id’ to a ‘discount_percent’. And also “gold” table, assigning a ‘surprise_gift’ to each specified ‘membership_id’. Finally, we will insert records individual records into the ‘review’ table, storing each ‘review’s id’, ‘rating’, ‘text’, and ‘submission date’.

remark: As a matter of fact, we inserted hundred(s) of instances in our database. However, we only show the first five rows of the instances in this report to reduce the space being used.

Code:

```
INSERT INTO customer (first_name, middle_name, last_name, email, phone_number,
date_of_birth, gender, registration_date, last_login_date, nationality)
-VALUES-
('Danielle', NULL, 'Johnson', 'danielle.johnson0@example.com', '33218196001', '1979-03-23',
'Male', '2021-01-22 06:23:12', '2025-10-31 23:51:06', 'Gabonese'),
('James', 'Debra', 'Gardner', 'james.gardner1@example.com', '79402654235', '1970-09-09', 'Male',
'2021-08-28 07:25:02', '2023-03-08 23:08:18', 'Samoan'),
('Melanie', 'Cassandra', 'Roman', 'melanie.roman2@example.com', '81618495931', '1997-06-10',
'Preferred not to say', '2023-04-18 22:03:05', '2023-08-20 20:54:42', 'British'),
('Melissa', 'Michelle', 'James', 'melissa.james3@example.com', '52553419283', '1970-08-28',
'Male', '2023-02-17 12:29:24', '2025-07-01 17:04:53', 'Togolese'),
('Jeffrey', NULL, 'Nguyen', 'jeffrey.nguyen4@example.com', '30564139537', '1954-07-23',
'Preferred not to say', '2023-02-01 18:37:06', '2023-09-06 16:24:50', 'Slovenian'),
```

```
INSERT INTO bronze (membership_ID, birthday_point)
-VALUES-
(1, 765),
(2, 330),
(3, 702),
(4, 160),
(5, 172),
```

```
INSERT INTO silver (membership_ID, discount_percent)
```

```
-VALUES-
```

```
(78, 24),
```

```
(167, 6),
```

```
(105, 28),
```

```
(30, 30),
```

```
(36, 26),
```

```
INSERT INTO gold (membership_ID, surprise_gift)
```

```
-VALUES-
```

```
(35, 'Promotion Code'),
```

```
(46, 'Promotion Code'),
```

```
(157, 'Airplane Ticket'),
```

```
(158, 'Airplane Ticket'),
```

```
(139, 'Airplane Ticket'),
```

```
INSERT INTO review (review_ID, rating, review_text, review_date)
```

```
-VALUES-
```

```
(1, 1, 'Agent every development say quality throughout.', '2024-06-18 01:52:41'),
```

```
(2, 2, 'Opportunity all behavior discussion.', '2022-01-16 01:52:41'),
```

```
(3, 5, 'Ago current practice nation determine operation speak according south recently future  
choice whatever from behavior.', '2025-08-28 01:52:41'),
```

```
(4, 1, 'Grow gas enough analysis least.', '2023-01-02 01:52:41'),
```

```
(5, 5, 'Win her need stop peace technology.', '2021-10-27 01:52:41'),
```

3. #parser_table3

We first load the order data by reading the file “orders_sampled_filtered_mapped.csv” from the “file location” directory and inserting its records into the “orders” table, skipping the first header row and specifically converting any empty string for the “days_since_prior_order” column into proper NULL values. Then, we will load the product data from “products_filtered_plus500_mapped_with_retailer.csv” in the same directory and insert those records into the “products” table, also skipping its header row. Similarly, we will populate the “order_products” table by importing data from “order_products_with_review_mapped_final.csv”. Finally, we will load the inventory data by reading the inventory_record.csv file from the “file location” directory and inserting its records into the inventory_record table, skipping the header row for both of these last two imports.

Code:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/orders_sampled_filtered.csv'

insert orders_sampled_filtered.csv data into orders table

LOAD DATA LOCAL INFILE

'D:/SIIT/Project_DB_Lecture/data_sampled_2/orders_sampled_filtered_mapped.csv'

INTO TABLE orders

FIELDS TERMINATED BY ','

OPTIONALLY ENCLOSED BY ''''

LINES TERMINATED BY '\r\n'

IGNORE 1 ROWS

(order_ID, customer_ID, eval_set, order_number, order_dow, order_hour_of_day, @days)

SET days_since_prior_order = NULLIF(TRIM(@days), "");

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/products_with_retailer.csv'

insert products_with_retailer.csv data into products table

LOAD DATA LOCAL INFILE

'D:/SIIT/Project_DB_Lecture/data_sampled_2/products_filtered_plus500_mapped_with_retailer.csv'

INTO TABLE products

FIELDS TERMINATED BY ','

OPTIONALLY ENCLOSED BY ''''

LINES TERMINATED BY '\n'

IGNORE 1 ROWS;

Location of file ->

'D:/SIIT/Project_DB_Lecture/data_sampled/order_products_combined_filtered.csv'

insert order_products_combined_filtered.csv data into order_products table

LOAD DATA LOCAL INFILE

'D:/SIIT/Project_DB_Lecture/data_sampled_2/order_products_with_review_mapped_final.csv'

INTO TABLE order_products

FIELDS TERMINATED BY ','

OPTIONALLY ENCLOSED BY ''''

```

LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

```

```

LOAD DATA LOCAL INFILE
'D:/SIIT/Project_DB_Lecture/data_random/inventory_record.csv'
INTO TABLE inventory_record
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

```

4. #parser_table4

We insert four new records into the “promotion” table, populating each with its unique promotion code, associated product ID, remaining quantity, description, discount details (type and value), and its active date range (start and end dates).

remark: As a matter of fact, we inserted hundred(s) of instances in our database. However, we only show the first five rows of the instances in this report to reduce the space being used.

Code:

```

INSERT INTO promotion (code, product_ID, remaining, description, discount_type,
discount_value, start_date, end_date)
VALUES
('PROMO0001', 913, 381, 'Agent every development say quality throughout.', 'Fixed Amount',
45.4, '2025-07-22 02:10:18', '2025-11-21 02:10:18'),
('PROMO0002', 205, 112, 'Opportunity all behavior discussion.', 'Fixed Amount', 9.09,
'2025-10-20 02:10:18', '2025-12-17 02:10:18'),
('PROMO0003', 2254, 78, 'Ago current practice nation determine operation speak according
south.', 'Fixed Amount', 31.84, '2025-07-26 02:10:18', '2025-11-30 02:10:18'),
('PROMO0004', 2007, 305, 'Future choice whatever from behavior benefit suggest page southern
role movie win her.', 'Fixed Amount', 48.7, '2025-10-30 02:10:18', '2025-12-28 02:10:18'),

```

5. #parser_table5

We first load the payment transaction data by reading the “payment_transaction.csv” file from the “file location” directory and inserting its records into the “payment_transaction” table, skipping the header row and trimming any extra whitespace from the “status” field. Then, we will load the invoice table; this process also skips the grader and converts any empty strings for the “note” and “updated_at” columns into proper NULL values. Finally, we will import the delivery data from “delivery.csv”, loading it into the “delivery” table while skipping its header, converting empty strings for “actual_delivery_date” into NULL, and trimming whitespace from the “status” field.

Code:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

```
LOAD DATA LOCAL INFILE
'D:/SIIT/Project_DB_Lecture/data_random/payment_transaction.csv'
INTO TABLE payment_transaction
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(payment_ID, order_ID, customer_ID, amount, payment_date, @status)
SET status = TRIM(@status);
```

```
LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_random/invoice.csv'
INTO TABLE invoice
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(invoice_ID,order_ID,customer_ID,invoice_date,due_date,total_amount,tax_amount,discount_a
mount,status,created_at,@updated_at,note)
SET
note = NULLIF(note, ""),
updated_at = NULLIF(@updated_at, "");
```

```

LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_random/delivery.csv'
INTO TABLE delivery
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(delivery_ID,order_ID,scheduled_date,@actual_delivery_date,@status)
SET
  actual_delivery_date = NULLIF(@actual_delivery_date, ""),
  status = TRIM(@status);

```

6. #parser_table6

We first insert data into the “address” table, adding five new address records, each complete with its associated “customer_id”, full street details contain of ‘house_number’, ‘street’, ‘city’, ‘province’, ‘postal_code’, ‘country’, and creation timestamp.

remark: As a matter of fact, we inserted hundred(s) of instances in our database. However, we only show the first five rows of the instances in this report to reduce the space being used.

Code:

```

INSERT INTO address (address_ID, customer_ID, house_number, street, city, province,
postal_code, country) VALUES
(1, 1, '545', 'Sangchuto Rd', 'Tha Muang', 'Kanchanaburi', '71110', 'Thailand'),
(2, 1, '645/2', 'Khlong Luang Rd', 'Khlong Luang', 'Pathum Thani', '12120', 'Thailand'),
(3, 1, '267/1', 'Chokchai-Dej Udom Rd', 'Nang Rong', 'Buri Ram', '31110', 'Thailand'),
(4, 2, '485/A', 'Si Ayutthaya Rd', 'Mueang Ayutthaya', 'Ayutthaya', '13000', 'Thailand'),
(5, 3, '619/1', 'Jerm Jom Phon Rd', 'Si Racha', 'Chonburi', '20110', 'Thailand'),

```

Random data record using Python

1. Customer

Customer_random code Explanation:

In this code, generates 300 fake customer records using the **Faker** library and Python's **random** module. The faker instance is seeded with 42 to make the results consistent every time the code runs.

We use the Faker library to generate data because the Faker library is used for generating text-type data, which aligns with the customer data we need.

Customer_random code:

```
from faker import Faker
import random
```

```
fake = Faker()
Faker.seed(42)
random.seed(42)
```

```
genders = ['Male', 'Female', 'Other', 'Preferred not to say']
```

```
nationalities = [
    "Afghan", "Albanian", "Algerian", "American", "Andorran", "Angolan",
    "Argentine", "Armenian", "Australian", "Austrian", "Azerbaijani",
    "Bahamian", "Bahraini", "Bangladeshi", "Barbadian", "Belarusian",
    "Belgian", "Belizean", "Beninese", "Bhutanese", "Bolivian",
    "Bosnian", "Brazilian", "British", "Bruneian", "Bulgarian",
    "Burkinabé", "Burmese", "Burundian", "Cambodian", "Cameroonian",
    "Canadian", "Cape Verdean", "Central African", "Chadian", "Chilean",
    "Chinese", "Colombian", "Comorian", "Congolese", "Costa Rican",
    "Croatian", "Cuban", "Cypriot", "Czech", "Danish", "Djiboutian",
    "Dominican", "Dutch", "East Timorese", "Ecuadorian", "Egyptian",
    "Emirati", "English", "Equatorial Guinean", "Eritrean", "Estonian",
    "Ethiopian", "Fijian", "Filipino", "Finnish", "French", "Gabonese",
    "Gambian", "Georgian", "German", "Ghanaian", "Greek", "Grenadian",
    "Guatemalan", "Guinean", "Guyanese", "Haitian", "Honduran",
    "Hungarian", "Icelandic", "Indian", "Indonesian", "Iranian",
    "Iraqi", "Irish", "Israeli", "Italian", "Ivorian", "Jamaican",
    "Japanese", "Jordanian", "Kazakhstani", "Kenyan", "Korean",
    "Kuwaiti", "Kyrgyzstani", "Laotian", "Latvian", "Lebanese",
```

"Liberian", "Libyan", "Lithuanian", "Luxembourgish", "Macedonian",
 "Malagasy", "Malawian", "Malaysian", "Maldivian", "Malian",
 "Maltese", "Marshallese", "Mauritanian", "Mauritian", "Mexican",
 "Micronesia", "Moldovan", "Monegasque", "Mongolian", "Montenegrin",
 "Moroccan", "Mozambican", "Namibian", "Nepalese", "New Zealander",
 "Nicaraguan", "Nigerian", "Nigerien", "Norwegian", "Omani",
 "Pakistani", "Palauan", "Palestinian", "Panamanian",
 "Papua New Guinean", "Paraguayan", "Peruvian", "Polish",
 "Portuguese", "Qatari", "Romanian", "Russian", "Rwandan",
 "Salvadoran", "Samoan", "San Marinese", "São Toméan",
 "Saudi Arabian", "Scottish", "Senegalese", "Serbian", "Seychellois",
 "Sierra Leonean", "Singaporean", "Slovak", "Slovenian",
 "Solomon Islander", "Somali", "South African", "Spanish",
 "Sri Lankan", "Sudanese", "Surinamese", "Swazi", "Swedish",
 "Swiss", "Syrian", "Taiwanese", "Tajikistani", "Tanzanian", "Thai",
 "Togolese", "Tongan", "Trinidadian", "Tunisian", "Turkish",
 "Turkmen", "Tuvaluan", "Ugandan", "Ukrainian", "Uruguayan",
 "Uzbekistani", "Venezuelan", "Vietnamese", "Welsh", "Yemeni",
 "Zambian", "Zimbabwean"

]

```
rows = []
for i in range(300):
    first_name = fake.first_name()
    middle_name = fake.first_name() if random.random() < 0.3 else None
    last_name = fake.last_name()
    email = f'{first_name.lower()}.{last_name.lower()}@example.com'
    phone_number = fake.msisdn()[1:]
    date_of_birth = fake.date_of_birth(minimum_age=18, maximum_age=80)
    gender = random.choice(genders)
    registration_date = fake.date_time_between(start_date="-5y", end_date="now")
    last_login_date = (
        fake.date_time_between(start_date=registration_date, end_date="now").strftime("%Y-%m-%d
%H:%M:%S")
        if random.random() < 0.8 else None
    )
    nationality = random.choice(nationalities)

    row = f'({first_name}, " \
        f'NULL' if middle_name is None else repr(middle_name)}, " \
```

```

f"{last_name}', '{email}', '{phone_number}', '{date_of_birth}', " \
f"{gender}', '{registration_date.strftime('%Y-%m-%d %H:%M:%S')}', " \
f"{'NULL' if last_login_date is None else repr(last_login_date)}, " \
f"{'nationality}')"
rows.append(row)

sql = "INSERT INTO customer (first_name, middle_name, last_name, email, phone_number,
date_of_birth, gender, registration_date, last_login_date, nationality)\nVALUES\n"
sql += ",\n".join(rows) + ";"

print(sql)

```

2. Loyalty_membership

Membership_random code Explanation:

This code generates loyalty membership data. It creates 200 membership records with different tier levels (Bronze, Silver, Gold) and realistic fields such as points balance, enrollment dates, and status. All members are inserted into the Bronze table, then 100 of them are randomly selected as Silver members, and 50 of those are further selected as Gold members. Finally, the script prints SQL INSERT statements for the four related tables: loyalty_membership, bronze, silver, and gold.

The reason we generated 200 loyalty membership records is because we have a total of 300 customers. Therefore, having 200 loyalty memberships is a number that makes sense and is realistically reasonable. 200 instances is sufficient for the loyalty_membership table, as having more instances would not impact the results.

Membership_random code:

```
import random
from datetime import datetime, timedelta

random.seed(42)

# Step 1: Generate 200 loyalty_membership entries
tier_levels = ['Bronze', 'Silver', 'Gold']
statuses = ['Active', 'Expired']
customer_ids = random.sample(range(1, 301), 200)

loyalty_rows = []
bronze_rows = []
silver_rows = []
gold_rows = []

for i, customer_id in enumerate(customer_ids):
    membership_id = i + 1
    tier = random.choice(tier_levels)
    points = random.randint(0, 10000)
    enrollment_date = datetime.now() - timedelta(days=random.randint(0, 1825))
    expiration_date = (
        enrollment_date + timedelta(days=random.randint(365, 1825))
        if random.random() < 0.7 else None
    )
```

```

status = 'Expired' if expiration_date and expiration_date < datetime.now() else 'Active'

loyalty_rows.append(
    f'({membership_id}, {customer_id}, {tier}, {points},
    '{expiration_date.strftime("%Y-%m-%d %H:%M:%S")}', "
    f'NULL' if expiration_date is None else repr(expiration_date.strftime("%Y-%m-%d
    %H:%M:%S"))}, {status})')
)
# Step 2: Bronze table (all loyalty_membership entries)
birthday_point = random.randint(100, 1000)
bronze_rows.append(f'({membership_id}, {birthday_point})')

# Step 3: Silver table (100 random bronze entries)
silver_ids = random.sample([row.split(",")[0][1:] for row in bronze_rows], 100)
for sid in silver_ids:
    discount = random.randint(5, 30)
    silver_rows.append(f'({sid}, {discount})')

# Step 4: Gold table (50 random silver entries)
gold_ids = random.sample([row.split(",")[0][1:] for row in silver_rows], 50)
gifts = ['Airplane Ticket', 'Gift Card', 'Promotion Code']
for gid in gold_ids:
    gift = random.choice(gifts)
    gold_rows.append(f'({gid}, {gift})')

# Final SQL
print("-- loyalty_membership")
print("INSERT INTO loyalty_membership (membership_ID, customer_ID, tier_level,
points_balance, enrollment_date, expiration_date, status)\nVALUES")
print(",\n".join(loyalty_rows) + ";")

print("\n-- bronze")
print("INSERT INTO bronze (membership_ID, birthday_point)\nVALUES")
print(",\n".join(bronze_rows) + ";")

print("\n-- silver")
print("INSERT INTO silver (membership_ID, discount_percent)\nVALUES")
print(",\n".join(silver_rows) + ";")

print("\n-- gold")

```

```
print("INSERT INTO gold (membership_ID, surprise_gift)\nVALUES")  
print("\n".join(gold_rows) + ";")
```

3. Retailor

Retailor_random code Explanation:

This code loads a product dataset and extracts unique retailer IDs. From those IDs, it randomly selects 150 retailers and generates fake retailer information using the Faker library, including status (Active/Inactive), account number, and account name. Finally, it formats the generated data into an SQL INSERT statement for the retailer table and prints it out.

Since our dataset contains a total of 300 customers, and customers can include both buyers and retailers — with some individuals acting as both — we decided to generate retailer data for 150 of them to better reflect a realistic scenario and it is sufficient for the retailer_random table, as having more instances would not impact the results.

Retailor_random code:

```
import pandas as pd

# Load the CSV
df =
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/products_filtered_plus500_mapped
_with_retailer.csv')

# Extract unique retailer_IDs
retailer_ids = df['retailer_id'].dropna().unique()
retailer_ids = [int(rid) for rid in retailer_ids]

import random
random.seed(42)

sampled_ids = random.sample(retailer_ids, 150)

from faker import Faker
fake = Faker()

rows = []
for rid in sampled_ids:
    status = random.choice(['Active', 'Inactive'])
    account_number = str(random.randint(1000000000, 9999999999))
    account_name = fake.company()
    rows.append(f'({rid}, '{status}', '{account_number}', '{account_name}'))

sql = "INSERT INTO retailer (retailer_id, status, account_number, account_name)\nVALUES\n"
```

```
sql += ",\n".join(rows) + ";"
```

```
print(sql)
```

4. Review

Review_random code Explanation:

- This code generates 951 reviews with ratings (1–5), optional review text, and dates. Using `random.seed(42)` and `Faker.seed(42)` ensures reproducibility, and the output is formatted as an SQL INSERT statement ready for the review table.
- Review text has an 80% chance of being generated
“`review_text = fake.sentence(...)` if `random.random() < 0.8` else `'NULL'`”
Simulates real-world cases where a review may have a rating but no text.
- Review date is randomized within the last 5 years
“`datetime.now() - timedelta(days=random.randint(0, 1825))`”
Adds time variability for trends and time-based analysis.

Review_random code:

```
import random
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()
random.seed(42)
Faker.seed(42)

reviews = []
for i in range(1, 951):
    rating = random.randint(1, 5)
    review_text = fake.sentence(nb_words=random.randint(5, 15)) if random.random() < 0.8 else 'NULL'
    review_date = datetime.now() - timedelta(days=random.randint(0, 1825))

    row = f'({i}, {rating}, " \
        f'{\'NULL\' if review_text == \'NULL\' else repr(review_text)}', " \
        f'{review_date.strftime(\'%Y-%m-%d %H:%M:%S\')})'
    reviews.append(row)

sql = "INSERT INTO review (review_ID, rating, review_text, review_date)\nVALUES\n"
sql += ",\n".join(reviews) + ";"

print(sql)
```

5. Inventory_record

inventory_record_review Code Explanation:

This code is designed to generate synthetic data records for the **inventory_record** table by randomly assigning values to each attribute. The **inventory_record** table has a **one-to-one relationship** with the **product** table, which originates from a Kaggle dataset. As such, the **product_id** attribute in the **inventory_record** table serves as a **foreign key (FK)** referencing the **product_id** in the **product** table. To maintain referential integrity between the two tables, the number of records generated in the **inventory_record** table must match the number of **product_id** entries in the **product** table, which totals **5084**. Each generated record must use a **product_id** that corresponds exactly to one found in the **product** table.

Code: for i in range(1, 5085):

This requirement is fulfilled by the line of code `for i in range(1, 5085):`, which iterates 5084 times to create 5084 records. Each iteration assigns the loop index `i` as the **product_id**, ensuring that the data in both tables remains consistent and properly linked.

Inventory_record_review Code:

```
import random
import csv
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()
random.seed(42)
Faker.seed(42)

# Open CSV file for writing
with open('inventory_record.csv', mode='w', newline="", encoding='utf-8') as file:
    writer = csv.writer(file)

    # Write header
    writer.writerow([
        'inventory_ID', 'product_ID', 'quantity_on_hand', 'quantity_reserved',
        'quantity_available', 'reorder_level', 'reorder_quantity',
        'last_restock_date', 'inventory_status', 'lot_number', 'storage_conditions'
```

)

Write 5084 rows

for i in range(1, 5085):

 quantity_on_hand = random.randint(0, 500)

 quantity_reserved = random.randint(0, quantity_on_hand)

 quantity_available = quantity_on_hand - quantity_reserved

 reorder_level = random.randint(10, 100)

 reorder_quantity = random.randint(50, 200)

 last_restock_date = (datetime.now() - timedelta(days=random.randint(0, 365))).strftime('%Y-%m-%d %H:%M:%S')

 inventory_status = (

 'Out of Stock' if quantity_on_hand == 0 else

 'backordered' if quantity_available < reorder_level else

 'In Stock'

)

 lot_number = fake.bathify(text='LOT-#####')

 storage_conditions = random.choice([

 'Cool and dry place', 'Refrigerated', 'Room temperature',

 'Keep away from sunlight', 'Humidity controlled'

])

writer.writerow([

 i, i, quantity_on_hand, quantity_reserved, quantity_available,

 reorder_level, reorder_quantity, last_restock_date,

 inventory_status, lot_number, storage_conditions

])

6. Promotion

promotion_random Code Explanation:

This code generates synthetic data records for the **promotion** table by randomly assigning values to each attribute. The **promotion** table has a **one-to-many (1:M)** relationship (product 1 : M promotion) with the **product** table, which is based on a dataset from Kaggle. As a result, the promotion table includes a **product_id** attribute that serves as a **foreign key (FK)** referencing the product table. In real-world business scenarios, promotions are typically applied to a limited number of products rather than the entire catalog. Given that the Kaggle dataset contains **5084 product_id**, the code randomly selects **100 product_id** from the range 1 to 5084 to simulate this selective promotion strategy.

The line of code **product_ids = random.sample(range(1, 5085), 100)** performs this sampling, ensuring 100 unique **product_id** are chosen.

Then, the line **for i, pid in enumerate(product_ids):** iterates through each selected **product_id**, using **i** as the promotion index and **pid** as the associated product. This loop is used to generate 100 promotion records, each linked to a different product.

promotion_random Code:

```
import random
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()
random.seed(42)
Faker.seed(42)

# Step 1: Sample 100 distinct product_IDs
product_ids = random.sample(range(1, 5085), 100)

# Step 2: Generate promotion rows
rows = []
for i, pid in enumerate(product_ids):
    code = f"PROMO{i+1:04d}"
    remaining = random.randint(0, 500)
    description = fake.sentence(nb_words=random.randint(5, 12))
    discount_type = random.choice(['Percentage', 'Fixed Amount'])
```

```

    discount_value = round(random.uniform(5, 50), 2) if discount_type == 'Fixed Amount' else
random.randint(5, 50)
    start_date = datetime.now() - timedelta(days=random.randint(0, 180))
    end_date = start_date + timedelta(days=random.randint(30, 180))

    row = f"('{code}', {pid}, {remaining}, '{description}', '{discount_type}', {discount_value}, " \
        f"'{start_date.strftime('%Y-%m-%d %H:%M:%S')}', '{end_date.strftime('%Y-%m-%d %H:%M:%S')}')"
    rows.append(row)

# Step 3: Build SQL
sql = "INSERT INTO promotion (code, product_ID, remaining, description, discount_type,
discount_value, start_date, end_date)\nVALUES\n"
sql += ",\n".join(rows) + ";"

print(sql)

```

7. Invoice

invoice_random Code Explanation:

This process involves generating data records for the **invoice** table by randomly assigning values to each attribute. The **invoice** table is considered a **weak entity**, meaning it cannot exist independently and must be associated with a parent entity—in this case, the **orders** table, which originates from a Kaggle dataset. The relationship between **invoice** and **orders** is **one-to-one (1:1)**, meaning each invoice corresponds to exactly one order, and each order has one invoice. Therefore, the **invoice** table includes the attribute **order_id** as a **foreign key (FK)** referencing the **orders** table.

Additionally, the **invoice** table has a **one-to-many (1:M)** relationship with the **customer** table, where each customer can have multiple invoices, but each invoice is linked to only one customer. As a result, the **invoice** table also includes the attribute **customer_id**, which serves as a foreign key referencing the **customer** table.

To maintain referential integrity, the number of invoice records must be equal to the number of order records, and each **order_id** used in the invoice table must match an existing **order_id** in the orders table. Likewise, the **customer_id** values in the invoice table must be consistent with those found in the orders table, which themselves originate from the **customer 1:M orders** relationship—where **customer_id** in the orders table is a foreign key referencing the customer table.

Invoice random Code:

```
import pandas as pd

import random

from faker import Faker

from datetime import datetime, timedelta

# Load the orders CSV

orders_df =
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/orders_sampled_filtered_mapped.csv')
```

```
# Normalize column names

orders_df.columns = orders_df.columns.str.strip().str.lower().str.replace(' ', '_')


# Filter valid rows with both order_id and customer_id

orders_df = orders_df.dropna(subset=['order_id', 'customer_id'])

orders_df['order_id'] = orders_df['order_id'].astype(int)

orders_df['customer_id'] = orders_df['customer_id'].astype(int)


# Initialize Faker

fake = Faker()

random.seed(42)

Faker.seed(42)


# Generate invoice data

records = []

for i, row in orders_df.iterrows():

    invoice_id = i + 1

    order_id = row['order_id']

    customer_id = row['customer_id']

    invoice_date = datetime.now() - timedelta(days=random.randint(0, 60))

    due_date = invoice_date + timedelta(days=random.randint(15, 45))

    total_amount = round(random.uniform(500, 10000), 2)

    tax_amount = round(total_amount * 0.07, 2)
```

```

discount_amount = round(random.uniform(0, total_amount * 0.1), 2) if random.random() <
0.5 else 0.0

status = random.choice(['Paid', 'Unpaid', 'Overdue'])

created_at = invoice_date

updated_at = invoice_date + timedelta(days=random.randint(0, 30)) if random.random() < 0.7
else ""

note = fake.sentence(nb_words=random.randint(5, 12)) if random.random() < 0.5 else ""

records.append([

    invoice_id, order_id, customer_id,

    invoice_date.strftime('%Y-%m-%d %H:%M:%S'),

    due_date.strftime('%Y-%m-%d %H:%M:%S'),

    total_amount, tax_amount, discount_amount,

    status, created_at.strftime('%Y-%m-%d %H:%M:%S'),

    updated_at if updated_at == "" else updated_at.strftime('%Y-%m-%d %H:%M:%S'),

    note

])

# Save to CSV

df_out = pd.DataFrame(records, columns=[

    'invoice_ID', 'order_ID', 'customer_ID', 'invoice_date', 'due_date',

    'total_amount', 'tax_amount', 'discount_amount', 'status',

    'created_at', 'updated_at', 'note'

])

df_out.to_csv('invoice.csv', index=False, encoding='utf-8')

```

8. Payment transaction

payment_transaction_random Code Explanation:

This program generates synthetic data records for the **payment_transaction** table by randomly assigning values to each attribute, while referencing real data from three existing tables:

order_products, **products**, and **invoice**. The purpose is to simulate payments made to retailers based on customer orders.

The **payment_transaction** table has a **one-to-many (1:M)** relationship with the **orders** table, meaning each order can be associated with multiple payment transactions. It also has a **one-to-many (1:M)** relationship with the **retailer** table, since each retailer can receive multiple payments. Therefore, the **payment_transaction** table includes the attribute **order_id** as a foreign key referencing the **orders** table, and the attribute **retailer_id** (labeled as **customer_ID** in the output) referencing the **retailer** table.

From the **invoice** table, the attribute **total_amount** is used to determine the total value of an order. In the **payment_transaction** table, this value is distributed into the attribute **amount**, which represents the portion of the payment allocated to each retailer linked to that order. The distribution is randomized but ensures that the sum of all amounts equals 70% of the invoice's total amount (to simulate platform fees or deductions).

payment_transaction_random Code:

```
import pandas as pd

import random

from faker import Faker

from datetime import datetime, timedelta

# Load CSVs

order_products_df =
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/order_products_with_review_mapped_final.csv')
```

```
products_df =
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/products_filtered_plus500_mapped
_with_retailer.csv')

invoice_df = pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_random/invoice.csv')

# Normalize column names

order_products_df.columns = order_products_df.columns.str.strip().str.lower().str.replace(' ', '_')
products_df.columns = products_df.columns.str.strip().str.lower().str.replace(' ', '_')
invoice_df.columns = invoice_df.columns.str.strip().str.lower().str.replace(' ', '_')

# Merge order_products with products to get retailer_id

merged_df = order_products_df.merge(products_df[['product_id', 'retailer_id']], on='product_id',
how='inner')

merged_df = merged_df.dropna(subset=['retailer_id'])

merged_df['retailer_id'] = merged_df['retailer_id'].astype(int)

# Merge with invoice to get total_amount

merged_df = merged_df.merge(invoice_df[['order_id', 'total_amount']], on='order_id', how='left')

merged_df = merged_df.dropna(subset=['total_amount'])

# Generate payment_transaction data

fake = Faker()

random.seed(42)

Faker.seed(42)
```

```

records = []

payment_id = 1

# Group by order_id
for order_id, group in merged_df.groupby('order_id'):

    total_amount = float(group['total_amount'].iloc[0])

    target_amount = round(total_amount * 0.7, 2)

    num_products = len(group)

    weights = [random.random() for _ in range(num_products)]

    total_weight = sum(weights)

    splits = [(w / total_weight) * target_amount for w in weights]

    splits[-1] = target_amount - sum(splits[:-1])

    splits = [round(x, 2) for x in splits]

    for i, (_, row) in enumerate(group.iterrows()):

        retailer_id = int(row['retailer_id'])

        payment_date = datetime.now() - timedelta(days=random.randint(0, 365))

        status = random.choice(['Successful', 'Failed', 'Pending'])

        records.append([

            payment_id, order_id, retailer_id, splits[i],

            payment_date.strftime('%Y-%m-%d %H:%M:%S'), status

```

```
)  
  
    payment_id += 1  
  
# Save to CSV  
  
df_out = pd.DataFrame(records, columns=[  
    'payment_ID', 'order_ID', 'customer_ID', 'amount',  
    'payment_date', 'status'  
)  
  
df_out.to_csv(r'D:/SIIT/Project_DB_Lecture/data_random/payment_transaction.csv',  
index=False, encoding='utf-8')
```

9. Delivery

Delivery_random code Explanation:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

```
“invoice_df = pd.read_csv(r'D:/SIIT/Project_DB_Lecture/invoice.csv’)”
```

This script generates delivery records based on existing invoice data. For each invoice, it creates a delivery with a scheduled date (1–10 days after the invoice)

```
“scheduled_date = row[‘invoice_date’] + timedelta(days=random.randint(1, 10))”
```

, a status (Pending, Shipped, Delivered, or Cancelled)

```
“status = random.choice([‘Pending’, ‘Shipped’, ‘Delivered’, ‘Cancelled’])”
```

, and an actual delivery date calculated according to the status.

```
“actual_delivery_date = "
    if status == ‘Delivered’:
        actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 5))
    elif status == ‘Shipped’:
        actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 3))
    elif status == ‘Cancelled’:
        actual_delivery_date = scheduled_date”
```

The resulting dataset is saved as delivery.csv for use in the database.

Delivery_random code:

```
import pandas as pd
import random
from datetime import datetime, timedelta

# Load invoice data
invoice_df = pd.read_csv(r'D:/SIIT/Project_DB_Lecture/invoice.csv')
```

```

# Normalize column names
invoice_df.columns = invoice_df.columns.str.strip().str.lower().str.replace(' ', '_')

# Filter valid order_IDs and invoice_dates
invoice_df = invoice_df.dropna(subset=['order_id', 'invoice_date'])
invoice_df['order_id'] = invoice_df['order_id'].astype(int)
invoice_df['invoice_date'] = pd.to_datetime(invoice_df['invoice_date'])

# Generate delivery records
random.seed(42)
records = []
for i, row in invoice_df.iterrows():
    delivery_id = i + 1
    order_id = row['order_id']
    scheduled_date = row['invoice_date'] + timedelta(days=random.randint(1, 10))
    status = random.choice(['Pending', 'Shipped', 'Delivered', 'Cancelled'])

    actual_delivery_date = ""
    if status == 'Delivered':
        actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 5))
    elif status == 'Shipped':
        actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 3))
    elif status == 'Cancelled':
        actual_delivery_date = scheduled_date

    records.append([
        delivery_id,
        order_id,
        scheduled_date.strftime('%Y-%m-%d %H:%M:%S'),
        actual_delivery_date.strftime('%Y-%m-%d %H:%M:%S') if actual_delivery_date else "",
        status
    ])

# Save to CSV
df_out = pd.DataFrame(records, columns=[
    'delivery_ID', 'order_ID', 'scheduled_date', 'actual_delivery_date', 'status'
])
df_out.to_csv('delivery.csv', index=False, encoding='utf-8')

```

10.Address

Address_random code Explanation:

This code generates realistic Thai addresses for 300 customers. Each customer can have 1–3 addresses randomly selected from a pool of 40 Thai districts (25 Mueang and 15 non-Mueang). House numbers are randomly generated. The output is formatted as an SQL INSERT statement for the address table, ready for database insertion.

Address_random code:

```
import random
from datetime import datetime

# Thai address pool (25 Mueang + 15 non-Mueang)
# ... [Insert the full 40-address list from earlier here] ...

thai_addresses = [
    # 25 Mueang Districts
    {"city": "Mueang Chiang Mai", "province": "Chiang Mai", "street": "Nimmanhaemin Rd",
    "postal_code": "50200", "country": "Thailand"},
    {"city": "Mueang Nakhon Ratchasima", "province": "Nakhon Ratchasima", "street":
    "Mittraphap Rd", "postal_code": "30000", "country": "Thailand"},
    {"city": "Mueang Khon Kaen", "province": "Khon Kaen", "street": "Sri Chan Rd",
    "postal_code": "40000", "country": "Thailand"},
    {"city": "Mueang Phuket", "province": "Phuket", "street": "Yaowarat Rd", "postal_code":
    "83000", "country": "Thailand"},
    {"city": "Mueang Chonburi", "province": "Chonburi", "street": "Sukhumvit Rd",
    "postal_code": "20000", "country": "Thailand"},
    {"city": "Mueang Udon Thani", "province": "Udon Thani", "street": "Prajak Rd",
    "postal_code": "41000", "country": "Thailand"},
    {"city": "Mueang Chiang Rai", "province": "Chiang Rai", "street": "Banpapraikan Rd",
    "postal_code": "57000", "country": "Thailand"},
    {"city": "Mueang Nakhon Pathom", "province": "Nakhon Pathom", "street": "Phetkasem Rd",
    "postal_code": "73000", "country": "Thailand"},
    {"city": "Mueang Surat Thani", "province": "Surat Thani", "street": "Talad Mai Rd",
    "postal_code": "84000", "country": "Thailand"},
    {"city": "Mueang Ayutthaya", "province": "Ayutthaya", "street": "Si Ayutthaya Rd",
    "postal_code": "13000", "country": "Thailand"},
```

```

{"city": "Mueang Sukhothai", "province": "Sukhothai", "street": "Sukhothai Rd",
"postal_code": "64000", "country": "Thailand"},
{"city": "Mueang Lampang", "province": "Lampang", "street": "Thipchang Rd",
"postal_code": "52000", "country": "Thailand"},
{"city": "Mueang Phitsanulok", "province": "Phitsanulok", "street": "Boromtrailokkanat Rd",
"postal_code": "65000", "country": "Thailand"},
{"city": "Mueang Nakhon Si Thammarat", "province": "Nakhon Si Thammarat", "street":
"Ratchadamnoen Rd", "postal_code": "80000", "country": "Thailand"},
{"city": "Mueang Trang", "province": "Trang", "street": "Ruangrit Rd", "postal_code":
"92000", "country": "Thailand"},
{"city": "Mueang Narathiwat", "province": "Narathiwat", "street": "Narathiwat Rd",
"postal_code": "96000", "country": "Thailand"},
{"city": "Mueang Pattani", "province": "Pattani", "street": "Yarang Rd", "postal_code":
"94000", "country": "Thailand"},
{"city": "Mueang Yala", "province": "Yala", "street": "Siri Rat Rd", "postal_code": "95000",
"country": "Thailand"},
{"city": "Mueang Nakhon Sawan", "province": "Nakhon Sawan", "street": "Sawanvitee Rd",
"postal_code": "60000", "country": "Thailand"},
{"city": "Mueang Ratchaburi", "province": "Ratchaburi", "street": "Phetkasem Rd",
"postal_code": "70000", "country": "Thailand"},
{"city": "Mueang Chanthaburi", "province": "Chanthaburi", "street": "Sukhumvit Rd",
"postal_code": "22000", "country": "Thailand"},
{"city": "Mueang Tak", "province": "Tak", "street": "Intharakiri Rd", "postal_code": "63000",
"country": "Thailand"},
{"city": "Mueang Kalasin", "province": "Kalasin", "street": "Pho Klang Rd", "postal_code":
"46000", "country": "Thailand"},
{"city": "Mueang Kamphaeng Phet", "province": "Kamphaeng Phet", "street":
"Ratchadamnoen Rd", "postal_code": "62000", "country": "Thailand"},
{"city": "Mueang Ubon Ratchathani", "province": "Ubon Ratchathani", "street": "Sapphasit
Rd", "postal_code": "34000", "country": "Thailand"},

```

15 Non-Mueang Districts

```

{"city": "Si Racha", "province": "Chonburi", "street": "Jerm Jom Phon Rd", "postal_code":
"20110", "country": "Thailand"},
{"city": "Bang Pa-in", "province": "Ayutthaya", "street": "U-Thong Rd", "postal_code":
"13160", "country": "Thailand"},
{"city": "Mae Sai", "province": "Chiang Rai", "street": "Phahonyothin Rd", "postal_code":
"57130", "country": "Thailand"},
{"city": "Hang Dong", "province": "Chiang Mai", "street": "Sanpatong Rd", "postal_code":
"50230", "country": "Thailand"},

```

```

{"city": "San Sai", "province": "Chiang Mai", "street": "Maejo Rd", "postal_code": "50210",
"country": "Thailand"},
{"city": "Ban Bueng", "province": "Chonburi", "street": "Chonburi-Ban Bueng Rd",
"postal_code": "20170", "country": "Thailand"},
{"city": "Tha Muang", "province": "Kanchanaburi", "street": "Sangchuto Rd", "postal_code":
"71110", "country": "Thailand"},
{"city": "Nang Rong", "province": "Buri Ram", "street": "Chokchai-Dej Udom Rd",
"postal_code": "31110", "country": "Thailand"},
{"city": "Phon", "province": "Khon Kaen", "street": "Phon-Chum Phae Rd", "postal_code":
"40120", "country": "Thailand"},
{"city": "Pak Chong", "province": "Nakhon Ratchasima", "street": "Thanarat Rd",
"postal_code": "30130", "country": "Thailand"},
{"city": "Sattahip", "province": "Chonburi", "street": "Sattahip-Nong Nooch Rd",
"postal_code": "20180", "country": "Thailand"},
{"city": "Ban Pong", "province": "Ratchaburi", "street": "Ban Pong Rd", "postal_code":
"70110", "country": "Thailand"},
{"city": "Phanom Sarakham", "province": "Chachoengsao", "street": "Sarakham Rd",
"postal_code": "24120", "country": "Thailand"},
{"city": "Khlong Luang", "province": "Pathum Thani", "street": "Khlong Luang Rd",
"postal_code": "12120", "country": "Thailand"},
{"city": "Khao Kho", "province": "Phetchabun", "street": "Khao Kho Viewpoint Rd",
"postal_code": "67270", "country": "Thailand"}
]

```

Generate realistic house numbers

```
def generate_house_number():
```

```
    return str(random.randint(1, 999)) + random.choice(["", "/1", "/2", "/A", "/B"])
```

Generate address rows for SQL

```
def generate_address_rows(num_customers=300, max_addresses_per_customer=3):
```

```
    rows = []
```

```
    address_id = 1
```

```
    for customer_id in range(1, num_customers + 1):
```

```
        num_addresses = random.randint(1, max_addresses_per_customer)
```

```
        for _ in range(num_addresses):
```

```
            addr = random.choice(thai_addresses)
```

```
            house_number = generate_house_number()
```

```
            row = f'({address_id}, {customer_id}, {house_number}, {addr['street']},
'{addr['city']}', '{addr['province']}', '{addr['postal_code']}', '{addr['country']}')'
```

```
        rows.append(row)
        address_id += 1
    return rows
# Generate SQL INSERT statement
def generate_sql_insert():
    rows = generate_address_rows()
    sql = "INSERT INTO address (address_ID, customer_ID, house_number, street, city,
province, postal_code, country) VALUES\n"
    sql += ",\n".join(rows) + ";"
    return sql

# Output the SQL
print(generate_sql_insert())
```

SQL queries

Query No.1	Product Quality Analysis with Review Ratings
Owner	<6622780409, Kittin Traisiwakul>
Objective	Identify top-rated and poorly-rated products based on customer reviews to support inventory, marketing, and quality control decisions.
Query	<pre> SELECT p.product_ID, p.product_name, ROUND(AVG(r.rating),2) AS avg_rating, COUNT(r.review_ID) AS review_count FROM products p JOIN order_products op ON p.product_ID = op.product_ID JOIN review r ON op.review_ID = r.review_ID GROUP BY p.product_ID, p.product_name HAVING review_count > 0 ORDER BY avg_rating DESC, review_count DESC LIMIT 10; </pre>

Result

product_ID	product_name	avg_rating ▼ 1	review_count ▼ 2
3200	Organic Cane Sugar	5.00	3
681	Organic Vanilla Extract	5.00	3
1765	Dark Chocolate Pretzels with Sea Salt	5.00	3
1162	Organic Sage	5.00	3
3680	Celery Sticks	5.00	3
3888	Ultra Soft Facial Tissues	5.00	3
4143	Oh My Yog! Organic Wild Quebec Blueberry Cream Top...	5.00	3
1186	Cocoa Noir Cold Brew Coffee With Almondmilk	5.00	2
1006	Organic Yokids Lemonade/Blueberry Variety Pack Yog...	5.00	2
3624	Pulp Free Orange Juice	5.00	2

Query No.2	Late Delivery Performance Tracking
Owner	<6622780409, Kittin Traisiwakul>
Objective	Monitor and analyze delivery timeliness trends over time to improve logistics operations, identify service quality issues, and maintain customer satisfaction.
Query	<pre> SELECT DATE_FORMAT(d.scheduled_date, '%Y-%m') AS delivery_month, COUNT(*) AS total_deliveries, SUM(CASE WHEN d.actual_delivery_date IS NOT NULL AND d.actual_delivery_date > d.scheduled_date THEN 1 ELSE 0 END) AS late_delivery_count, ROUND(100 * SUM(CASE WHEN d.actual_delivery_date IS NOT NULL AND d.actual_delivery_date > d.scheduled_date THEN 1 ELSE 0 END) / COUNT(*), 2) AS late_delivery_percentage FROM delivery d WHERE d.status = 'Delivered' GROUP BY DATE_FORMAT(d.scheduled_date, '%Y-%m') ORDER BY delivery_month DESC; </pre>

Result

delivery_month ▼ 1	total_deliveries	late_delivery_count	late_delivery_percentage
2025-11	26	26	100.00
2025-10	140	140	100.00
2025-09	92	92	100.00

Query No.3	Products Frequently Bought Together
Owner	<6622780409, Kittin Traisiwakul>
Objective	Identify product pairs commonly purchased in the same order to optimize cross-selling strategies, product placement, and bundling opportunities.
Query	<pre> SELECT p1.product_ID AS product_1_id, p1.product_name AS product_1, p2.product_ID AS product_2_id, p2.product_name AS product_2, d.department, COUNT(DISTINCT op1.order_ID) AS times_bought_together, ROUND(AVG(i.total_amount), 2) AS avg_order_value FROM order_products op1 JOIN order_products op2 ON op1.order_ID = op2.order_ID AND op1.product_ID < op2.product_ID JOIN products p1 ON op1.product_ID = p1.product_ID JOIN products p2 ON op2.product_ID = p2.product_ID AND p1.department_ID = p2.department_ID JOIN departments d ON p1.department_ID = d.department_ID JOIN invoice i ON op1.order_ID = i.order_ID GROUP BY p1.product_ID, p1.product_name, p2.product_ID, p2.product_name, d.department HAVING times_bought_together >= 5 ORDER BY times_bought_together DESC LIMIT 10; </pre>

Result

product_1_id	product_1	product_2_id	product_2	department	times_bought_together	avg_order_value
1241	Bag of Organic Bananas	4362	Organic Hass Avocado	produce	26	5996.59
1241	Bag of Organic Bananas	1980	Organic Strawberries	produce	23	4918.72
1241	Bag of Organic Bananas	2611	Organic Raspberries	produce	18	5082.56
2061	Organic Baby Spinach	2351	Banana	produce	16	5330.80
1980	Organic Strawberries	2351	Banana	produce	15	5087.75
1241	Bag of Organic Bananas	2061	Organic Baby Spinach	produce	14	6408.19
1561	Strawberries	2351	Banana	produce	14	6191.76
2351	Banana	4394	Large Lemon	produce	13	6667.46
2461	Limes	4394	Large Lemon	produce	13	5061.05
2351	Banana	4362	Organic Hass Avocado	produce	12	6166.87

Query No.4	Products Never Ordered
Owner	<6622780409, Kittin Traisiwakul>
Objective	Identify products in the catalog that have never been purchased to optimize inventory management, reduce carrying costs, and improve product portfolio performance.
Query	<pre> SELECT p.product_ID, p.product_name, d.department FROM products p LEFT JOIN order_products op ON p.product_ID = op.product_ID LEFT JOIN departments d ON p.department_ID = d.department_ID WHERE op.order_ID IS NULL ORDER BY p.product_ID LIMIT 10; </pre>

Result

product_ID	product_name	department
4585	Shaving Blade Cartridges	personal care
4586	Organic Russet Potatoes	produce
4587	Fresh Brush Heavy Duty Refills Toilet Bowl	household
4588	All Natural Unbleached Waxed Paper	pantry
4589	Pink Lemonade	beverages
4590	Fit Triple Berry Crunch Whole Grain Granola	breakfast
4591	Apple & Maple Chicken Mini Links	meat seafood
4592	Naturally Sun Dried Organic Raisins	snacks
4593	Natural Laundry Stain Remover, Free & Clear	household
4594	Strawberry Raspberry Blended Greek Yogurt	dairy eggs

Query No.5	Customer Spending by Gender and Age Group
Owner	<6622770442, Kanade Areepoonsiri>
Objective	-- Shows total spending patterns grouped by customer demographics -- Identifies which demographic groups spend the most
Query	<pre> SELECT c.gender, CASE WHEN FLOOR(DATEDIFF(NOW(), c.date_of_birth) / 365) < 25 THEN 'Under 25' WHEN FLOOR(DATEDIFF(NOW(), c.date_of_birth) / 365) < 35 THEN '25-34' WHEN FLOOR(DATEDIFF(NOW(), c.date_of_birth) / 365) < 50 THEN '35-49' ELSE '50+' END AS age_group, COUNT(DISTINCT c.customer_ID) AS customer_count, ROUND(AVG(total_spent.lifetime_value), 2) AS avg_total_spending, ROUND(AVG(total_spent.order_count), 2) AS avg_orders, ROUND(AVG(total_spent.lifetime_value) / NULLIF(AVG(total_spent.order_count), 0), 2) AS avg_per_order FROM customer c LEFT JOIN (SELECT i.customer_ID, SUM(i.total_amount) AS lifetime_value, COUNT(DISTINCT i.order_ID) AS order_count FROM invoice i GROUP BY i.customer_ID) AS total_spent ON c.customer_ID = total_spent.customer_ID GROUP BY c.gender, age_group HAVING customer_count >= 5 ORDER BY avg_total_spending DESC LIMIT 10; </pre>

Result

gender	age_group	customer_count	avg_total_spending ▼ 1	avg_orders	avg_per_order
Female	35-49	18	29341.23	5.89	4982.46
Preferred not to say	35-49	17	29062.61	5.50	5284.11
Preferred not to say	25-34	15	28933.09	5.20	5564.06
Male	35-49	24	27238.89	4.94	5509.04
Other	25-34	19	26857.07	5.85	4593.94
Male	Under 25	8	26762.92	5.00	5352.58
Female	50+	35	26752.88	5.00	5350.58
Other	50+	31	26540.17	4.95	5361.65
Female	25-34	9	26284.07	5.13	5128.60
Female	Under 25	7	24247.38	5.00	4849.48

Query No.6	Inventory Stock Levels vs Recent Demand
Owner	<6622770442, Kanade Areepoonsiri>
Objective	-- Compares current inventory quantities to recent order demand -- Shows which products may need restocking soon -- Identify products running low on stock -- Find products with excess inventory
Query	<pre> SELECT p.product_ID, p.product_name, ir.quantity_on_hand, ir.reorder_level, COUNT(DISTINCT op.order_ID) AS orders_last_90_days, ROUND(COUNT(DISTINCT op.order_ID) / 90.0, 2) AS avg_daily_orders, ROUND(ir.quantity_on_hand / NULLIF(COUNT(DISTINCT op.order_ID) / 90.0, 0), 1) AS days_of_stock_remaining, CASE WHEN ir.quantity_on_hand / NULLIF(COUNT(DISTINCT op.order_ID) / 90.0, 0) < 15 THEN 'Low Stock' WHEN ir.quantity_on_hand / NULLIF(COUNT(DISTINCT op.order_ID) / 90.0, 0) > 60 THEN 'Overstocked' ELSE 'Adequate' END AS stock_status FROM inventory_record ir JOIN products p ON ir.product_ID = p.product_ID LEFT JOIN order_products op ON p.product_ID = op.product_ID WHERE ir.inventory_status = 'In Stock' GROUP BY p.product_ID, p.product_name, ir.quantity_on_hand, ir.reorder_level HAVING orders_last_90_days > 0 ORDER BY days_of_stock_remaining ASC LIMIT 10; </pre>

Result

product_ID	product_name	quantity_on_hand	reorder_level	orders_last_90_days	avg_daily_orders	days_of_stock_remaining	▲ 1	stock_status
2351	Banana	254	38	119	1.32	192.1		Overstocked
1241	Bag of Organic Bananas	331	28	128	1.42	232.7		Overstocked
4362	Organic Hass Avocado	273	55	72	0.80	341.3		Overstocked
1980	Organic Strawberries	315	10	82	0.91	345.7		Overstocked
2061	Organic Baby Spinach	354	90	81	0.90	393.3		Overstocked
2017	Organic Creamy Tomato Soup	20	16	4	0.04	450.0		Overstocked
2576	Organic Lacinato (Dinosaur) Kale	66	16	11	0.12	540.0		Overstocked
414	Shredded Parmesan	98	45	14	0.16	630.0		Overstocked
4488	Canola Oil	48	11	6	0.07	720.0		Overstocked
3335	Organic Unsweetened Almond Milk	198	92	24	0.27	742.5		Overstocked

Query No.7	Product Ratings by Department
Owner	<6622770442, Kanade Areepoonsiri>
Objective	-- Shows average customer ratings for products with reviews -- Identifies well-rated vs poorly-rated products -- Find products needing quality improvement -- Highlight highly-rated products for marketing
Query	<pre> SELECT p.product_ID, p.product_name, d.department, COUNT(DISTINCT r.review_ID) AS review_count, ROUND(AVG(r.rating), 2) AS avg_rating, ROUND(100 * SUM(CASE WHEN r.rating <= 2 THEN 1 ELSE 0 END) / COUNT(*), 2) AS low_rating_percentage, COUNT(DISTINCT op.order_ID) AS total_orders, CASE WHEN AVG(r.rating) >= 4.5 THEN 'Excellent' WHEN AVG(r.rating) >= 4.0 THEN 'Good' WHEN AVG(r.rating) >= 3.0 THEN 'Average' ELSE 'Needs Improvement' END AS rating_category FROM products p JOIN order_products op ON p.product_ID = op.product_ID JOIN review r ON op.review_ID = r.review_ID LEFT JOIN departments d ON p.department_ID = d.department_ID GROUP BY p.product_ID, p.product_name, d.department HAVING review_count >= 10 ORDER BY avg_rating DESC LIMIT 10; </pre>

Result

product_ID	product_name	department	review_count	avg_rating	low_rating_percentage	total_orders	rating_category
1833	Organic Russet Potato	produce	11	4.18	9.09	11	Good
1284	Organic Snipped Green Beans	produce	10	4.00	10.00	10	Good
557	Clementines	produce	12	3.92	8.33	12	Average
3340	Organic Fat Free Milk	dairy eggs	10	3.70	30.00	10	Average
774	Broccoli Crown	produce	10	3.70	10.00	10	Average
3148	Organic Italian Parsley Bunch	produce	27	3.67	22.22	27	Average
2170	Organic Yellow Onion	produce	27	3.63	22.22	27	Average
2820	Organic Cucumber	produce	26	3.62	26.92	26	Average
3121	Total 2% with Strawberry Lowfat Greek Strained Yog...	dairy eggs	10	3.60	30.00	10	Average
2543	Organic Black Beans	canned goods	12	3.58	33.33	12	Average

Query No.8	Monthly Sales by Department
Owner	<6622770442, Kanade Areepoonsiri>
Objective	-- Shows sales revenue and order volume for each department by month -- Identifies seasonal patterns in different product categories -- Track department performance over time -- Identify peak months for each department
Query	SELECT d.department, MONTH(i.invoice_date) AS month_number, MONTHNAME(i.invoice_date) AS month_name, COUNT(DISTINCT op.order_ID) AS order_count, ROUND(SUM(i.total_amount), 2) AS total_revenue, ROUND(AVG(i.total_amount), 2) AS avg_order_value FROM departments d JOIN products p ON d.department_ID = p.department_ID JOIN order_products op ON p.product_ID = op.product_ID JOIN orders o ON op.order_ID = o.order_ID JOIN invoice i ON o.order_ID = i.order_ID GROUP BY d.department, MONTH(i.invoice_date), MONTHNAME(i.invoice_date) LIMIT 10;

Result

department	month_number	month_name	order_count	total_revenue	avg_order_value
alcohol	9	September	9	50637.52	4219.79
alcohol	10	October	7	65979.29	5075.33
babies	9	September	20	198814.49	3206.69
babies	10	October	22	274772.91	4040.78
babies	11	November	1	8758.99	8758.99
bakery	9	September	122	827344.13	5170.90
bakery	10	October	145	1092596.62	5329.74
bakery	11	November	3	15178.72	5059.57
beverages	9	September	209	1991755.03	5133.39
beverages	10	October	207	2010854.22	5182.61

Query No.9	Inventory Turnover Analysis
Owner	<6622780318, Jetnipat Wanichsakul>
Objective	Analyze product inventory efficiency by measuring how quickly products sell relative to stock levels, enabling data-driven inventory management and capital optimization.
Query	<pre> SELECT p.product_ID, p.product_name, COUNT(DISTINCT op.order_ID) AS orders_with_product, IFNULL(ROUND(AVG(ir.quantity_on_hand),2),0) AS avg_quantity_on_hand, CASE WHEN AVG(ir.quantity_on_hand) > 0 THEN ROUND(COUNT(DISTINCT op.order_ID) / AVG(ir.quantity_on_hand), 4) ELSE NULL END AS turnover_proxy, CASE WHEN COUNT(DISTINCT op.order_ID) / NULLIF(AVG(ir.quantity_on_hand), 0) >= 1.0 THEN 'High' WHEN COUNT(DISTINCT op.order_ID) / NULLIF(AVG(ir.quantity_on_hand), 0) >= 0.5 THEN 'Medium' WHEN COUNT(DISTINCT op.order_ID) / NULLIF(AVG(ir.quantity_on_hand), 0) IS NOT NULL THEN 'Low' ELSE 'No Data' END AS turnover_category FROM products p LEFT JOIN order_products op ON p.product_ID = op.product_ID LEFT JOIN inventory_record ir ON p.product_ID = ir.product_ID GROUP BY p.product_ID, p.product_name ORDER BY turnover_proxy DESC LIMIT 10; </pre>

Result

product_ID	product_name	orders_with_product	avg_quantity_on_hand	turnover_proxy	turnover_category
1831	Spring Water	26	6.00	4.3333	High
2928	Extra Fancy Unsalted Mixed Nuts	4	1.00	4.0000	High
2986	Dijon Mustard	3	1.00	3.0000	High
1292	Organic Sweet Potatoes	3	1.00	3.0000	High
4494	Globe Eggplant	8	3.00	2.6667	High
2216	Sweet Corn On The Cob	4	2.00	2.0000	High
2629	Organic Fuji Apple	29	16.00	1.8125	High
3806	Lemonade	9	5.00	1.8000	High
2604	Organic Whole Milk	38	25.00	1.5200	High
641	Low Fat Plain Yogurt	4	3.00	1.3333	High

Query No.10	Monthly Customer Order Summary With Retention
Owner	<6622780318, Jetnipat Wanichsakul>
Objective	Analyze customer acquisition cohorts and their purchasing behavior over time to measure retention effectiveness, identify high-value registration periods, and optimize customer lifecycle management.
Query	<pre> SELECT DATE_FORMAT(c.registration_date, '%Y-%m') AS cohort_month, COUNT(DISTINCT c.customer_ID) AS total_customers, COUNT(DISTINCT CASE WHEN o.order_ID IS NOT NULL THEN c.customer_ID END) AS customers_with_orders, ROUND(100 * COUNT(DISTINCT CASE WHEN o.order_ID IS NOT NULL THEN c.customer_ID END) / COUNT(DISTINCT c.customer_ID), 2) AS purchase_rate, ROUND(AVG(i.total_amount), 2) AS avg_invoice_amount FROM customer c LEFT JOIN orders o ON c.customer_ID = o.customer_ID LEFT JOIN invoice i ON o.order_ID = i.order_ID GROUP BY DATE_FORMAT(c.registration_date, '%Y-%m') ORDER BY cohort_month DESC LIMIT 10; </pre>

Result

cohort_month ▼ 1	total_customers	customers_with_orders	purchase_rate	avg_invoice_amount
2025-10	8	4	50.00	3988.73
2025-09	6	4	66.67	4674.47
2025-08	7	6	85.71	6138.68
2025-07	8	5	62.50	6095.24
2025-06	5	3	60.00	5307.19
2025-05	6	3	50.00	3845.53
2025-04	3	1	33.33	3399.83
2025-03	5	3	60.00	5468.01
2025-02	5	2	40.00	5700.51
2025-01	4	2	50.00	6320.74

Query No.11	Loyalty Membership Tier Summary
Owner	<6622780318, Jetnipat Wanichsakul>
Objective	Analyze loyalty program performance across membership tiers to evaluate tier value and member engagement
Query	<pre> SELECT lm.tier_level, COUNT(DISTINCT lm.customer_ID) AS total_members, ROUND(AVG(lm.points_balance), 2) AS avg_points_balance, ROUND(AVG(DATEDIFF(NOW(), lm.enrollment_date)), 0) AS avg_days_as_member, ROUND(AVG(customer_value.total_spent), 2) AS avg_total_spent, ROUND(100 * SUM(CASE WHEN lm.status = 'Active' THEN 1 ELSE 0 END) / COUNT(*), 2) AS active_percentage FROM loyalty_membership lm LEFT JOIN (SELECT i.customer_ID, SUM(i.total_amount) AS total_spent FROM invoice i GROUP BY i.customer_ID) AS customer_value ON lm.customer_ID = customer_value.customer_ID GROUP BY lm.tier_level ORDER BY FIELD(lm.tier_level, 'Gold', 'Silver', 'Bronze') LIMIT 10; </pre>

Result

tier_level	total_members	avg_points_balance	avg_days_as_member	avg_total_spent	active_percentage
Gold	67	5049.52	953	23872.67	79.10
Silver	68	4728.81	903	25906.77	73.53
Bronze	65	5030.60	853	25266.84	76.92

Query No.12	Customer Lifetime Value Ranking
Owner	<6622780318, Jetnipat Wanichsakul>
Objective	Identify and rank customers by their total historical spending to prioritize retention efforts, personalize marketing, and allocate resources toward high-value customers.
Query	<pre> SELECT i.customer_ID, CONCAT(c.first_name,' ',c.last_name) AS customer_name, ROUND(SUM(i.total_amount),2) AS lifetime_value, COUNT(DISTINCT i.order_ID) AS invoice_count FROM invoice i JOIN customer c ON i.customer_ID = c.customer_ID GROUP BY i.customer_ID ORDER BY lifetime_value DESC LIMIT 10; </pre>

Result

customer_ID	customer_name	lifetime_value	1	invoice_count
116	Katelyn Mitchell	63712.84		10
138	Joseph Glass	60026.18		11
23	Daniel Hamilton	58751.92		11
93	David Preston	58017.86		10
32	Brent Gray	57002.81		8
164	David Dalton	53909.96		9
104	Anna Owens	53161.29		8
148	Tammy Hogan	52823.93		9
25	Ashley Sullivan	49984.89		8
132	Jose Hunt	49174.97		7

Query No.13	Loyalty Program Impact Analysis
Owner	<6622781852, Pisit Pongpan>
Objective	Compares average invoice value and invoice volume by loyalty tier (plus non-members) to show how membership correlates with spending. Measures loyalty program ROI and identifies high-value tiers for targeting. Informs tier-specific marketing, offers, and resource allocation.
Query	<pre> SELECT lm.tier_level AS tier, ROUND(AVG(i.total_amount),2) AS avg_invoice_amount, COUNT(DISTINCT i.invoice_ID) AS invoice_count FROM loyalty_membership lm JOIN customer c ON lm.customer_ID = c.customer_ID JOIN orders o ON c.customer_ID = o.customer_ID JOIN invoice i ON o.order_ID = i.order_ID WHERE lm.status = 'Active' GROUP BY lm.tier_level UNION ALL SELECT 'Non-member', ROUND(AVG(i2.total_amount),2), COUNT(DISTINCT i2.invoice_ID) FROM customer c2 LEFT JOIN loyalty_membership lm2 ON c2.customer_ID = lm2.customer_ID JOIN orders o2 ON c2.customer_ID = o2.customer_ID JOIN invoice i2 ON o2.order_ID = i2.order_ID WHERE lm2.membership_ID IS NULL ORDER BY avg_invoice_amount DESC LIMIT 10; </pre>

Result

tier	avg_invoice_amount ▼ 1	invoice_count
Non-member	5407.75	266
Bronze	5085.91	168
Silver	5007.74	164
Gold	4947.20	186

Query No.14	Delivery Performance by Province
Owner	<6622781852, Pisit Pongpan>
Objective	Shows delivery speed and success rate for each province and identifies provinces with delivery issues.
Query	<pre> SELECT a.province, COUNT(DISTINCT d.delivery_ID) AS total_deliveries, ROUND(AVG(DATEDIFF(d.actual_delivery_date, d.scheduled_date)), 2) AS avg_delay_days, ROUND(100 * SUM(CASE WHEN d.actual_delivery_date <= d.scheduled_date THEN 1 ELSE 0 END) / COUNT(*), 2) AS on_time_percentage, ROUND(100 * SUM(CASE WHEN d.status = 'Cancelled' THEN 1 ELSE 0 END) / COUNT(*), 2) AS cancellation_percentage, COUNT(DISTINCT o.customer_ID) AS unique_customers FROM address a JOIN customer c ON a.customer_ID = c.customer_ID JOIN orders o ON c.customer_ID = o.customer_ID JOIN delivery d ON o.order_ID = d.order_ID WHERE d.actual_delivery_date IS NOT NULL GROUP BY a.province HAVING total_deliveries >= 10 ORDER BY on_time_percentage ASC LIMIT 10; </pre>

Result

province	total_deliveries	avg_delay_days	on_time_percentage	1	cancellation_percentage	unique_customers
Nakhon Si Thammarat	34	1.94	23.53		23.53	11
Ayutthaya	62	1.91	24.24		24.24	19
Phuket	49	1.78	24.49		24.49	9
Tak	44	2.05	25.00		25.00	9
Ratchaburi	82	1.80	25.61		25.61	22
Khon Kaen	65	1.83	27.69		27.69	18
Yala	34	1.56	29.41		29.41	7
Phitsanulok	37	1.76	29.73		29.73	10
Lampang	44	1.72	29.79		29.79	10
Pathum Thani	57	1.79	29.82		29.82	14

Query No.15	Slow-Moving Inventory Identification
Owner	<6622781852, Pisit Pongpan>
Objective	Identifies products with low recent sales relative to stock levels, and highlights inventory that may need clearance.
Query	<pre> SELECT * FROM (SELECT p.product_ID, p.product_name, d.department, ir.quantity_on_hand, ir.last_restock_date, DATEDIFF(NOW(), ir.last_restock_date) AS days_since_restock, COUNT(DISTINCT op.order_ID) AS orders_in_last_90_days, CASE WHEN COUNT(DISTINCT op.order_ID) = 0 AND DATEDIFF(NOW(), ir.last_restock_date) > 180 THEN 'Critical' WHEN COUNT(DISTINCT op.order_ID) <= 2 AND DATEDIFF(NOW(), ir.last_restock_date) > 120 THEN 'High Risk' WHEN COUNT(DISTINCT op.order_ID) <= 5 AND DATEDIFF(NOW(), ir.last_restock_date) > 90 THEN 'Medium Risk' ELSE 'Low Risk' END AS slow_moving_risk FROM inventory_record ir JOIN products p ON ir.product_ID = p.product_ID LEFT JOIN departments d ON p.department_ID = d.department_ID LEFT JOIN order_products op ON p.product_ID = op.product_ID WHERE ir.inventory_status = 'In Stock' GROUP BY p.product_ID, p.product_name, d.department, ir.quantity_on_hand, ir.last_restock_date) AS inv WHERE inv.slow_moving_risk IN ('Critical', 'High Risk') ORDER BY FIELD(inv.slow_moving_risk, 'Critical', 'High Risk'), inv.days_since_restock DESC LIMIT 10; </pre>

Result

product_ID	product_name	department	quantity_on_hand	last_restock_date	days_since_restock	orders_in_last_90_days	slow_moving_risk
4991	Veggie Pepperoni Pizza	deli	364	2024-11-09 02:05:23	378	0	Critical
4855	Organic Date & Lemon Artisan Crisps Made With Fres...	missing	348	2024-11-11 02:05:23	376	0	Critical
5014	Salisbury Steak with macaroni and cheese Salisbury...	frozen	382	2024-11-11 02:05:23	376	0	Critical
4693	Monterey Jack Sliced Cheese	dairy eggs	347	2024-11-11 02:05:23	376	0	Critical
4858	Strawberry on the Bottom Nonfat Greek Yogurt	dairy eggs	292	2024-11-11 02:05:23	376	0	Critical
5044	White Paper Towel Roll	household	123	2024-11-13 02:05:23	374	0	Critical
4907	Multigrain Onion & Garlic Quinoa Crisps	snacks	124	2024-11-13 02:05:23	374	0	Critical
4765	Lemon Lime Sparkling Energy Water	beverages	361	2024-11-13 02:05:23	374	0	Critical
4869	Israeli Couscous with Mushrooms	dry goods pasta	80	2024-11-15 02:05:23	372	0	Critical
4838	Tropical Splash Moisturizing Shave Gel	personal care	458	2024-11-19 02:05:23	368	0	Critical

Query No.16	Departments Purchased Together
Owner	<6622781852, Pisit Pongpan>
Objective	Shows which departments are bought together in the same orders and identifies complementary product categories.
Query	<pre> SELECT d1.department AS department_1, d2.department AS department_2, COUNT(DISTINCT o.order_ID) AS orders_with_both_departments, COUNT(DISTINCT o.customer_ID) AS unique_customers, ROUND(AVG(i.total_amount), 2) AS avg_order_total FROM orders o JOIN order_products op1 ON o.order_ID = op1.order_ID JOIN products p1 ON op1.product_ID = p1.product_ID JOIN departments d1 ON p1.department_ID = d1.department_ID JOIN order_products op2 ON o.order_ID = op2.order_ID JOIN products p2 ON op2.product_ID = p2.product_ID JOIN departments d2 ON p2.department_ID = d2.department_ID AND d1.department_ID < d2.department_ID LEFT JOIN invoice i ON o.order_ID = i.order_ID GROUP BY d1.department, d2.department HAVING orders_with_both_departments >= 20 ORDER BY orders_with_both_departments DESC LIMIT 10; </pre>

Result

department_1	department_2	orders_with_both_departments	unique_customers	avg_order_total
produce	dairy eggs	533	187	5623.17
produce	snacks	319	153	5332.60
dairy eggs	snacks	319	162	5480.02
produce	beverages	306	150	5388.77
beverages	dairy eggs	305	155	5430.52
produce	pantry	290	145	5233.39
frozen	produce	286	152	5260.66
pantry	dairy eggs	278	148	5390.00
frozen	dairy eggs	277	153	5007.46
beverages	snacks	231	140	5353.20

Query No.17	Customer Value Analysis with Invoice Data
Owner	<6622781969, Thinnaphop Direkpok>
Objective	-- Shows total number of orders placed by each customer -- Displays the average invoice amount per customer for revenue insight -- Helps identify high-value customers based on order volume and spending -- Supports customer segmentation by purchasing behavior -- Tracks overall customer activity and transaction patterns over time
Query	<pre> SELECT c.customer_ID, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, COUNT(DISTINCT o.order_ID) AS orders_count, IFNULL(ROUND(AVG(i.total_amount),2), 0) AS avg_invoice_amount FROM customer c LEFT JOIN orders o ON c.customer_ID = o.customer_ID LEFT JOIN invoice i ON o.order_ID = i.order_ID GROUP BY c.customer_ID, c.first_name, c.last_name ORDER BY orders_count DESC LIMIT 10; </pre>

Result

customer_ID	customer_name	orders_count	avg_invoice_amount
138	Joseph Glass	11	5456.93
23	Daniel Hamilton	11	5341.08
159	Sydney Floyd	10	4202.62
93	David Preston	10	5801.79
89	Danielle Dean	10	4715.77
98	Calvin Daniels	10	3599.60
116	Katelyn Mitchell	10	6371.28
119	Shari Smith	9	3006.71
164	David Dalton	9	5990.00
190	John Scott	9	4826.14

Query No.18	Customer Activity Segmentation
Owner	<6622781969, Thinnaphop Direkpok>
Objective	-- Measures customer recency by calculating days since the last invoice -- Identifies customer activity level using rule-based segmentation -- Shows total orders and total spending for each customer -- Helps detect high-value and high-engagement customers -- Supports churn analysis by flagging inactive or low-activity customers -- Enables targeted marketing and personalized customer retention strategies
Query	<pre> SELECT c.customer_ID, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, DATEDIFF(NOW(), MAX(i.invoice_date)) AS days_since_last_order, COUNT(DISTINCT o.order_ID) AS total_orders, ROUND(SUM(i.total_amount), 2) AS total_spent, CASE WHEN DATEDIFF(NOW(), MAX(i.invoice_date)) <= 30 AND COUNT(DISTINCT o.order_ID) >= 5 THEN 'Very Active' WHEN DATEDIFF(NOW(), MAX(i.invoice_date)) <= 60 AND COUNT(DISTINCT o.order_ID) >= 3 THEN 'Active' WHEN DATEDIFF(NOW(), MAX(i.invoice_date)) <= 90 THEN 'Moderate' WHEN DATEDIFF(NOW(), MAX(i.invoice_date)) > 180 THEN 'Inactive' ELSE 'Low Activity' END AS activity_segment FROM customer c LEFT JOIN orders o ON c.customer_ID = o.customer_ID LEFT JOIN invoice i ON o.order_ID = i.order_ID GROUP BY c.customer_ID, c.first_name, c.last_name HAVING total_orders > 0 ORDER BY total_spent DESC LIMIT 10; </pre>

Result

customer_ID	customer_name	days_since_last_order	total_orders	total_spent ▼ 1	activity_segment
116	Katelyn Mitchell	25	10	63712.84	Very Active
138	Joseph Glass	25	11	60026.18	Very Active
23	Daniel Hamilton	29	11	58751.92	Very Active
93	David Preston	36	10	58017.86	Active
32	Brent Gray	31	8	57002.81	Active
164	David Dalton	36	9	53909.96	Active
104	Anna Owens	35	8	53161.29	Active
148	Tammy Hogan	26	9	52823.93	Very Active
25	Ashley Sullivan	25	8	49984.89	Very Active
132	Jose Hunt	28	7	49174.97	Very Active

Query No.19	Promotion Effectiveness Summary
Owner	<6622781969, Thinnaphop Direkpok>
Objective	-- Shows promotion details linked to each product, including discount type and value -- Measures promotional impact by counting total orders containing the promoted product -- Evaluates average order value during promotion periods -- Helps identify which promotions drive higher product demand -- Supports analysis of promotion effectiveness for future campaign planning
Query	<pre> SELECT pr.code AS promo_code, pr.product_ID, p.product_name, pr.discount_type, pr.discount_value, pr.start_date, pr.end_date, COUNT(DISTINCT op.order_ID) AS total_orders_with_product, ROUND(AVG(i.total_amount), 2) AS avg_order_value FROM promotion pr JOIN products p ON pr.product_ID = p.product_ID LEFT JOIN order_products op ON pr.product_ID = op.product_ID LEFT JOIN invoice i ON op.order_ID = i.order_ID GROUP BY pr.code, pr.product_ID, p.product_name, pr.discount_type, pr.discount_value, pr.start_date, pr.end_date ORDER BY total_orders_with_product DESC LIMIT 10; </pre>

Result

promo_code	product_ID	product_name	discount_type	discount_value	start_date	end_date	total_orders_with_product	avg_order_value
PROMO0014	768	Apple Honeycrisp Organic	Fixed Amount	45.48	2025-07-09 02:10:18	2025-09-07 02:10:18	34	5100.51
PROMO0099	2170	Organic Yellow Onion	Fixed Amount	25.92	2025-06-03 02:10:18	2025-09-09 02:10:18	27	4843.05
PROMO0044	356	Whole Milk	Fixed Amount	6.98	2025-08-08 02:10:18	2025-10-10 02:10:18	13	6675.18
PROMO0009	713	Organic Sticks Low Moisture Part Skim Mozzarella S...	Fixed Amount	5.88	2025-08-01 02:10:18	2025-11-17 02:10:18	11	3921.51
PROMO0003	2254	Sour Cream	Fixed Amount	31.84	2025-07-26 02:10:18	2025-11-30 02:10:18	6	4956.48
PROMO0031	2788	Shredded Mozzarella	Percentage	43	2025-10-15 02:10:18	2026-01-13 02:10:18	6	5452.96
PROMO0084	459	Spaghetti No 12	Percentage	34	2025-09-30 02:10:18	2026-03-15 02:10:18	6	5163.06
PROMO0096	3242	Salted Butter	Fixed Amount	28.97	2025-05-16 02:10:18	2025-09-17 02:10:18	5	4066.45
PROMO0015	1792	Gluten Free Multigrain Bread	Percentage	26	2025-10-27 02:10:18	2026-04-25 02:10:18	4	7101.93
PROMO0047	1023	Organic Cream Cheese Bar	Fixed Amount	31.21	2025-09-25 02:10:18	2026-02-12 02:10:18	4	6274.80

Query No.20	Active Promotion Effectiveness
Owner	<6622781969, Thinnaphop Direkpok>
Objective	-- Lists active promotions currently within their valid date range -- Shows how many orders include products that are under promotion -- Helps identify which active promotions are driving the highest order volume -- Supports real-time monitoring of promotion performance -- Enables quick assessment of effective vs. low-performing promotions
Query	<pre> SELECT pr.code, pr.product_ID, p.product_name, pr.start_date, pr.end_date, COUNT(DISTINCT op.order_ID) AS orders_with_promo_product FROM promotion pr JOIN products p ON pr.product_ID = p.product_ID LEFT JOIN order_products op ON pr.product_ID = op.product_ID LEFT JOIN orders o ON op.order_ID = o.order_ID WHERE op.order_ID IS NOT NULL AND pr.start_date <= NOW() AND pr.end_date >= NOW() GROUP BY pr.code, pr.product_ID, p.product_name, pr.start_date, pr.end_date ORDER BY orders_with_promo_product DESC LIMIT 10; </pre>

Result

code	product_ID	product_name	start_date	end_date	orders_with_promo_product	1
PROMO0084	459	Spaghetti No 12	2025-09-30 02:10:18	2026-03-15 02:10:18	6	
PROMO0003	2254	Sour Cream	2025-07-26 02:10:18	2025-11-30 02:10:18	6	
PROMO0031	2788	Shredded Mozzarella	2025-10-15 02:10:18	2026-01-13 02:10:18	6	
PROMO0015	1792	Gluten Free Multigrain Bread	2025-10-27 02:10:18	2026-04-25 02:10:18	4	
PROMO0047	1023	Organic Cream Cheese Bar	2025-09-25 02:10:18	2026-02-12 02:10:18	4	
PROMO0025	3680	Celery Sticks	2025-09-20 02:10:18	2026-02-01 02:10:18	3	
PROMO0029	1308	Organic Pumpkin	2025-10-18 02:10:18	2025-11-29 02:10:18	3	
PROMO0036	838	Broccoli & Cheddar Bake Meal Bowl	2025-07-07 02:10:18	2025-12-28 02:10:18	3	
PROMO0048	3101	Crackers Almond Flour Farmhouse Sharp Cheddar	2025-10-22 02:10:18	2026-02-20 02:10:18	2	
PROMO0050	4523	Tall Kitchen Drawstring Bags	2025-09-17 02:10:18	2026-01-30 02:10:18	2	

Query No.21	Unpaid Invoice Detection
Owner	<6622780151, Patipat Permsub>
Objective	Find customer orders without a paid invoice to identify revenue that has been generated but not yet collected. Helping finance and accounting teams track, improve the billing process, and manage cash flow more effectively.
Query	<pre> SELECT DISTINCT o.customer_ID, CONCAT(c.first_name,' ',c.last_name) AS customer_name, o.order_ID FROM orders o JOIN customer c ON o.customer_ID = c.customer_ID LEFT JOIN invoice i ON o.order_ID = i.order_ID AND i.status = 'Paid' WHERE i.invoice_ID IS NULL ORDER BY o.customer_ID LIMIT 10; </pre>

Result

customer_ID	customer_name	order_ID
1	Danielle Johnson	307
1	Danielle Johnson	349
1	Danielle Johnson	389
1	Danielle Johnson	464
2	James Gardner	199
2	James Gardner	347
3	Melanie Roman	110
3	Melanie Roman	183
3	Melanie Roman	747
3	Melanie Roman	820

Query No.22	Product Popularity by Order Frequency
Owner	<6622780151, Patipat Permsub>
Objective	To rank products by the number of orders they appear in (Order Frequency). It counts how many times each product is included in customer orders, groups results by product and department, and returns the top 30 most frequently ordered items. The purpose is to identify the most popular products across departments for better inventory planning and marketing strategies.
Query	<pre> SELECT p.product_ID, p.product_name, d.department, COUNT(op.order_ID) AS orders_containing_product FROM products p JOIN order_products op ON p.product_ID = op.product_ID LEFT JOIN departments d ON p.department_ID = d.department_ID GROUP BY p.product_ID, p.product_name, d.department ORDER BY orders_containing_product DESC LIMIT 10; </pre>

Result

product_ID	product_name	department	orders_containing_product	▼ 1
1241	Bag of Organic Bananas	produce	128	
2351	Banana	produce	119	
1980	Organic Strawberries	produce	82	
2061	Organic Baby Spinach	produce	81	
4362	Organic Hass Avocado	produce	72	
4394	Large Lemon	produce	56	
4407	Organic Avocado	produce	49	
2461	Limes	produce	44	
2611	Organic Raspberries	produce	42	
1561	Strawberries	produce	39	

Query No.23	Delivery Timeliness vs Review Ratings
Owner	<6622780151, Patipat Permsub>
Objective	Used to analyze the relationship between delivery timeliness and review ratings, by comparing ratings across different categories of on-time and late deliveries. It helps distinguish whether the ratings reflect only product quality or are also influenced by the delivery experience.
Query	<pre> SELECT CASE WHEN DATEDIFF(d.actual_delivery_date, d.scheduled_date) <= 0 THEN 'On Time' WHEN DATEDIFF(d.actual_delivery_date, d.scheduled_date) <= 2 THEN '1-2 Days Late' WHEN DATEDIFF(d.actual_delivery_date, d.scheduled_date) <= 5 THEN '3-5 Days Late' ELSE 'Over 5 Days Late' END AS delivery_category, COUNT(DISTINCT d.delivery_ID) AS total_deliveries, ROUND(AVG(r.rating), 2) AS avg_review_rating, ROUND(100 * SUM(CASE WHEN r.rating >= 4 THEN 1 ELSE 0 END) / COUNT(*), 2) AS high_rating_percentage, COUNT(DISTINCT o.customer_ID) AS unique_customers FROM delivery d JOIN orders o ON d.order_ID = o.order_ID JOIN order_products op ON o.order_ID = op.order_ID JOIN review r ON op.review_ID = r.review_ID WHERE d.status = 'Delivered' AND d.actual_delivery_date IS NOT NULL GROUP BY delivery_category ORDER BY FIELD(delivery_category, 'On Time', '1-2 Days Late', '3-5 Days Late', 'Over 5 Days Late');</pre>

Result

delivery_category	total_deliveries	avg_review_rating	high_rating_percentage	unique_customers
1-2 Days Late	111	3.13	40.28	83
3-5 Days Late	147	3.07	43.83	102

Query No.24	Retailer Sales Performance Summary
Owner	<6622780151, Patipat Permsub>
Objective	Provides a sales performance summary of retailers, showing products supplied, orders, revenue, average order value, and product ratings. It also categorizes retailers into tiers (Top, Mid, Standard) based on revenue Helping identify top performers and support strategic business decisions.
Query	<pre> SELECT r.retailer_ID, CONCAT(c.first_name, ' ', c.last_name) AS retailer_name, r.account_name, COUNT(DISTINCT p.product_ID) AS products_supplied, COUNT(DISTINCT op.order_ID) AS total_orders, ROUND(SUM(i.total_amount), 2) AS total_revenue, ROUND(AVG(i.total_amount), 2) AS avg_order_value, ROUND(AVG(rev.rating), 2) AS avg_product_rating, CASE WHEN SUM(i.total_amount) >= 50000 THEN 'Top Tier' WHEN SUM(i.total_amount) >= 20000 THEN 'Mid Tier' ELSE 'Standard' END AS retailer_tier FROM retailer r JOIN customer c ON r.retailer_ID = c.customer_ID JOIN products p ON r.retailer_ID = p.retailer_ID JOIN order_products op ON p.product_ID = op.product_ID JOIN orders o ON op.order_ID = o.order_ID JOIN invoice i ON o.order_ID = i.order_ID LEFT JOIN review rev ON op.review_ID = rev.review_ID GROUP BY r.retailer_ID, c.first_name, c.last_name, r.account_name HAVING total_orders >= 5 ORDER BY total_revenue DESC LIMIT 10; </pre>

Result

retailer_ID	retailer_name	account_name	products_supplied	total_orders	total_revenue	avg_order_value	avg_product_rating	retailer_tier
201	Joseph Butler	Johnson-Davis	30	163	949746.27	5554.07	2.94	Top Tier
228	Heather Thompson	Ford-Smith	32	168	942870.99	5069.20	2.97	Top Tier
35	Laura Knight	Vega Inc	22	153	854748.27	5444.26	3.08	Top Tier
256	Robert Golden	Williams-Kelly	29	131	698205.32	4951.81	3.04	Top Tier
254	Donna Baker	Savage-Wade	35	120	658666.88	5186.35	3.37	Top Tier
217	Kimberly Decker	Black, Castro and Jackson	29	113	654944.63	5412.77	3.02	Top Tier
210	James Wilson	Chavez Ltd	30	113	644658.28	5241.12	3.09	Top Tier
229	Devon Morrison	Bennett, Burton and Adams	45	106	629034.41	5517.85	3.01	Top Tier
271	James Rodgers	Ruiz, Morrison and Peterson	34	106	618497.57	5241.50	2.92	Top Tier
235	Jeffrey Jones	Clark Inc	37	97	586743.37	5696.54	3.15	Top Tier

References

- Supermarket / Superstore Dataset Bundle, Kaggle [Supermarket / Superstore Dataset Bundle](#)
- Tangworakitthaworn, Preecha. CSS325 Project 1 Assignment Brief (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 3: Data Modeling Part 1 Business Requirement Analysis and Data Model (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 4: ER Diagram (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 5: EER Diagram (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 6,7: Relational Data Model and Mapping ERD to Relational Database Schema (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 8: Keys and FDs (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 9: Normalization (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 10: SQLPart1 DDL DML (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 11: SQLPart3 Basic QL Std (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 12: Advance QL, Set Operation, Correlated and Nested Sub Query (PDF provided).